Volume V

Software Architecture & The 5-Block Invariant Protocol

CreativeWorkzStudio LLC / ProfessorSeanEX

Version 1.0.0, 2026-11-07

Table of Contents

1. Volume Synopsis . . . 1
2. Chapter 1: OmniCode 5-Block Architectural Framework . . . 2
3. Architecture Overview . . . 3
4. 1. The Universal 5-Block Code Structure . . . 4
5. 2. Core Mechanics & State Invariants . . . 5
    5.1. The Twin Relics: Shield and Sword . . . 5
    5.2. Invariant Axioms . . . 5
6. 3. Multi-Phase Engine Evolution . . . 6
7. 4. Layer 2 Presentation Architecture & 5-Screen Navigation . . . 7
8. Chapter 2: Dual-Frontend Presentation Layer (CLI + Native X11) . . . 8
9. Abstract . . . 9
10. 1. The Core Architectural Philosophy . . . 10
    10.1. Universal Action Dispatcher & Intent Emitter . . . 10
11. 2. Native Desktop GUI Architecture: Why Not Web? . . . 12
    11.1. Double-Buffering & Render Synchronization . . . 12
    11.2. Dynamic Visual Palette: The Twin Relics . . . 13
12. 3. The 5-Block Implementation Mapping . . . 14
    12.1. Presentation Architecture Breakdown (`include/gui/gui.h` & `src/gui/gui.c`) . . . 14
13. 4. Headless CI & Deterministic Replay . . . 15
14. 5. Custom Engine Evolution Roadmap . . . 16
15. Chapter 3: Central Manifest & Pre-Flight Diagnostic Integrity . . . 17
16. Specification Abstract & Purpose . . . 18
17. 1. Expected Architecture: The Central Manifest Pattern . . . 19
    17.1. 1.1 Structural Hierarchy . . . 19
    17.2. 1.2 The Diagnostic Severity Spectrum . . . 19
    17.3. 1.3 Expected Pre-Flight Engine States . . . 20
18. 2. Architectural Metadata Dimensions & Pedagogical Balance . . . 21
    18.1. 2.1 Striking the Balance: Beginner-Accessible yet Senior-Rigorous . . . 21
    18.2. 2.2 Explicit Temporal Stamping & Version History . . . 21
    18.3. 2.3 The 5-Dimension Compiled Schema (`module_metadata_t`) . . . 21
    18.4. 2.4 Active Dependency Graph Enforcement . . . 22
19. 3. Expected Pre-Flight "Loose Bolts" System Checks . . . 23
20. 4. Target Verification Criteria ("Expected" Baseline) . . . 24
21. 5. Actual Results vs. Expected Gap Analysis . . . 25
    21.1. 5.1 Empirical Benchmark Telemetry . . . 25
    21.2. 5.2 How to Reproduce and Verify Every Result . . . 26
    21.3. 5.3 Gap Analysis & Next Architectural Horizons . . . 27
22. Chapter 4: Layer 0 Core Utilities Code Tour . . . 28

23. Overview . . . 29
24. 1. Architectural Philosophy & Mission . . . 30
24.1. Key Architectural Pillars . . . 30
25. 2. Subsystem Layout & File Manifest . . . 31
26. 3. Core Engine Cogs & API Tour . . . 32
26.1. 3.1 Bounded Math & Saturation Arithmetic (`math_utils.[ch]`) . . . 32
26.2. 3.2 2D AABB Geometry & Layout Tripwires (`geom_utils.[ch]`) . . . 32
26.3. 3.3 Monotonic Time & Clock Pacing (`time_utils.[ch]`, `clock_pacer.[ch]`) . . . 33
26.4. 3.4 Bounded String Safety (`string_utils.[ch]`) . . . 33
26.5. 3.5 Data-Driven Formula Evaluator (`formula_eval.[ch]`) . . . 34
26.6. 3.6 Universal Engine Orchestrators . . . 34
27. 4. Verification & Testing Matrix . . . 35
28. Chapter 5: Layer 0 Zero-Heap I/O Pipeline Subsystem . . . 36
29. Overview . . . 37
30. 1. Architectural Philosophy & Mission . . . 38
30.1. Key Architectural Pillars . . . 38
31. 2. Subsystem Layout & File Manifest . . . 39
32. 3. Core Engine Cogs & API Tour . . . 40
32.1. 3.1 Filesystem Utilities (`fs_utils.[ch]`) . . . 40
32.2. 3.2 Path Resolution Engine (`path_finder.[ch]`) . . . 40
32.3. 3.3 Zero-Heap JSON Tokenizer & Parser (`json_utils.[ch]`) . . . 40
32.4. 3.4 Pre-Flight Master Loader & DAG Auditor (`master_loader.[ch]`) . . . 41
33. 4. Verification & Testing Matrix . . . 42
34. Chapter 6: Layer 0 Categorized Flight Logging Subsystem . . . 43
35. Overview . . . 44
36. 1. Architectural Philosophy & Mission . . . 45
36.1. Key Architectural Pillars . . . 45
37. 2. Subsystem Layout & File Manifest . . . 46
38. 3. Core Engine Cogs & API Tour . . . 47
38.1. 3.1 Severity Taxonomy & Public Macros (`log.h`) . . . 47
38.2. 3.2 In-Memory Flight Recorder Ring Buffer (`log_engine.[ch]`) . . . 47
38.3. 3.3 Log Rotation & Disk Archiving . . . 48
38.4. 3.4 Quarantined Fallback Isolation (`log_fallback.[ch]`) . . . 48
39. 4. Verification & Testing Matrix . . . 49
40. Chapter 7: Layer 0 State Debugger & Diagnostic Engine . . . 50
41. Overview . . . 51
42. 1. Architectural Philosophy & Mission . . . 52
42.1. Key Architectural Pillars . . . 52
43. 2. Subsystem Layout & File Manifest . . . 53
44. 3. Core Engine Cogs & API Tour . . . 54
44.1. 3.1 State Representation & Frozen Snapshot Ring (`debug_engine.[ch]`) . . . 54

44.2. 3.2 Deterministic FNV-1a Hashing . . . 54
44.3. 3.3 Multi-Format Forensic Exporters . . . 54
44.4. 3.4 Live ASCII Inspection Cards . . . 55
44.5. 3.5 Active Invariant Tripwire Matrix . . . 55
44.6. 3.6 Interactive REPL & State Rewind (`--debug`) . . . 56
45. 4. Verification & Testing Matrix . . . 57
46. Chapter 8: Layer 1 Simulation Engine Code Tour . . . 58
47. Overview . . . 59
48. 1. State Structure & Memory Layout (`player_state_t`) . . . 60
49. 2. Universal Action Dispatcher (`game_process_command`) . . . 61
49.1. Execution Pipeline: . . . 61
50. 3. Diagnostic Self-Auditing (`game_audit_state`) . . . 62
51. Chapter 9: Layer 2 Presentation Viewport Code Tour . . . 63
52. Overview . . . 64
53. 1. The Movie Reel Mental Model & Subfolder Strata . . . 65
54. 2. The Movie Reel: Horizontal Motion & Vertical Depth . . . 67
54.1. 2.1 Horizontal Axis: Push and Pull (Left and Right) . . . 67
54.2. 2.2 Vertical Axis: Pop Out and Sink In (Up and Down) . . . 67
55. 3. The 2.5D Tactical Isometric Engine . . . 68
55.1. Isometric Projection & Unprojection Reciprocity: . . . 68
56. 4. Non-Anonymous Type Invariant Covenant . . . 69
57. 5. Verification Telemetry . . . 70
58. Chapter 10: Layer 0/2 Audio Event Bus & Mixer Subsystem . . . 71
59. Overview . . . 72
60. 1. Architectural Philosophy: The Decoupled Audio Machine . . . 73
60.1. Axiomatic Invariants: . . . 73
61. 2. Subsystem File Layout & Header Structure . . . 74
62. 3. Multi-Channel Mixer Architecture . . . 75
63. 4. Non-Anonymous Ontological Types . . . 76
64. 5. Verification Telemetry . . . 77
65. Chapter 11: Layer 4 Verification & Test Suite Code Tour . . . 78
66. Overview . . . 79
67. 1. The Four-Tier Testing Hierarchy . . . 80
67.1. Tier 1: Functional Debugging (`tests/functional/`) . . . 80
67.2. Tier 2: Subsystem Unit Harnesses (`tests/unit/`) . . . 80
67.3. Tier 3: Integrated Utilities Harness (`tests/integrated/`) . . . 81
67.4. Tier 4: System Verification (`tests/system/`) . . . 81
68. 2. The Four Ontological Testing Sections . . . 82
69. 3. CMake & CTest Test Targets . . . 83
70. 4. Execution & Verification Commands . . . 84
71. Chapter 12: Technical Specification: Data-Driven Architecture & Manifest System . . . 85

72. Abstract & Architectural Mandate . . . 86
73. 1. System Philosophy: Manifest as Main Menu . . . 87
73.1. 1.1 The Menu-to-Settings Ontological Pattern . . . 87
73.2. 1.2 OmniCode 5-Block Conformance in JSON Serialization . . . 87
74. 2. Active Data-Driven Subsystems . . . 89
74.1. 2.1 Logging & Telemetry (`data/log/`) . . . 89
74.2. 2.2 Mathematical Continuum (`data/math/`) . . . 89
74.3. 2.3 Monotonic Timing & Frame Pacing (`data/time/`) . . . 89
74.4. 2.4 Lexical Buffers & Token Dictionary (`data/string/`) . . . 89
75. 3. The Fallback Tripwire Protocol . . . 90
76. 4. Verification & Diagnostic Testing . . . 91

Chapter 1. Volume Synopsis

System boundaries, layered strata, dual-frontend GUI/CLI, and pre-flight zero-heap diagnostic manifests.

Chapter 2. Chapter 1: OmniCode 5-Block Architectural Framework

Source File: /home/professorseanex/Tenebris and Lux/docs/architecture/ARCHITECTURE.adoc

Chapter 3. Architecture Overview

This document specifies the technical design, subsystem boundaries, data models, and ontological lifecycle governing **Tenebris and Lux** ("Darkness and Light"). Set within the chivalric **Kingdom of Sacyra** (pronounced **"saw-kee-ruh"**, /s□□□ki□r□/), the game is a **Tactical Strategy RPG (TRPG)** centered on dual protagonists—**Elijah Ro-enheart** (son of former Head of the Royal Guard) and **Lady Katrina Jaren** (noble court aspirant seeking the Adventurers' Guild)—who wield the primordial **Shield of Lux** and **Sword of Tenebris** against the dread legions of **The Dark Lord** (the ancient Libiran sovereign Usuru).

The engine strictly separates **simulation state** from **presentation** across 5 strata:

```
Layer 4: Verification Suite       [ tests/ ]  (19 Automated CTest Targets)
            |
Layer 3: Application Entry        [ src/main.c ]  (Presentation Router & CLI)
            |
Layer 2: Presentation & Viewport  [ src/gui/ <-> include/gui/ ]  (X11 Double-Buffered)
            |
Layer 1: Simulation Engine        [ src/sim/ <-> include/sim/ ]  (Core, Combat, Items,
World)
            |
Layer 0: Core Utilities           [ src/util/ <-> include/util/ ] (Math, Geom, Data,
Log)
```

Chapter 4. 1. The Universal 5-Block Code Structure

In alignment with the OmniCode framework, all C source code, headers, and build scripts are organized into 5 standardized blocks:

1. **BLOCK 1: METADATA BLOCK**: Defines module identity, layer, author, version, role, and invariant axioms.
2. **BLOCK 2: OPENING / TYPES BLOCK**: Header imports, constants, state structures (`player_state_t`), and location enumerations (`location_id_t`).
3. **BLOCK 3: BODY BLOCK**: Core simulation routines, relic stance transitions, combat calculations, and moral alignment resolution.
4. **BLOCK 4: AUDIT PROCEDURES BLOCK**: Self-healing diagnostic routines, invariant tripwires, and verification harnesses.
5. **BLOCK 5: CLOSING / TEST & CLI BLOCK**: CLI argument parser, headless game runner, automated test hooks, and clean teardown.

Chapter 5. 2. Core Mechanics & State Invariants

5.1. The Twin Relics: Shield and Sword

- **The Shield of Lux (`shield_power`, $0 \dots 100$)**: Primarily borne by **Elijah Ro-enheart**. Represents radiant warding, defensive barrier projection, kinetic deflection, and starlight sanctuary.
- **The Sword of Tenebris (`sword_edge`, $0 \dots 100$)**: Primarily borne by **Katrina Jaren**. Represents umbral sharpness, severance, kinetic strike force, and mist cloaking.
- **Tactical Stance (`combat_stance_t`)**:
  - `STANCE_NEUTRAL`: Relics held in readiness.
  - `STANCE_WARDING`: Shield raised forward; mitigates incoming damage.
  - `STANCE_STRIKING`: Sword drawn for offensive severance; boosts strike damage.
  - `STANCE_EQUILIBRIUM`: Twin mastery; balances offensive and defensive resonance.
- **Moral Sway (`moral_sway`, $-100 \dots +100$)**:
  - Positive (> 0): **Virtue & Mercy** (using power to protect the defenseless).
  - Negative (< 0): **Tyranny & Ruin** (using power for cruel domination and ruin).

5.2. Invariant Axioms

1. **Shield Power Boundary**: $0 \le \text{state.shield_power} \le \text{MAX_RESONANCE}$.
2. **Sword Edge Boundary**: $0 \le \text{state.sword_edge} \le \text{MAX_RESONANCE}$.
3. **Moral Sway Bounded Range**: $-100 \le \text{state.moral_sway} \le +100$.
4. **Valid Location Invariant**: $0 \le \text{state.location} < \text{LOC_COUNT}$.
5. **Deterministic Replay Guarantee**: Any identical sequence of actions from a given initial state must produce identical state telemetry.

Chapter 6. 3. Multi-Phase Engine Evolution

Phase	Milestone	Subsystem Architecture
Phase 1 (Complete)	`v0.0.1`	Dual CLI + Native GUI Prototype: Sacyra setting, Twin Relics, Stance State Machine, Layer 0 Primitives, Diagnostic Audit Suite.
Phase 2 (Active)	`v0.1.0`	Tactical Strategy RPG Engine & 5-Screen UI: 2.5D responsive scaling (3x zoom), TrueType vector fonts (Cinzel, MedievalSharp), 32-slot inventory, provincial topology, and 19 CTest targets.
Phase 3	`v0.2.0`	Cinematic Combat Resolution & AI: Real-time kinetic deflection windows, poise breaking, dynamic skill execution from data catalogs, and enemy flanking AI.
Phase 4	`v1.0.0`	Full TRPG Campaign of Sacyra: Dynamic party management, persistent disk save/load, King’s Curse midpoint planar wound fork, and the 5 canonical ending routes.

Chapter 7. 4. Layer 2 Presentation Architecture & 5-Screen Navigation

The presentation engine is organized into modular sub-assemblies under `src/gui/`: * `gui_palette`: High Fantasy Hero's Journey color palette (Radiant Gold, Tenebrous Violet). * `gui_fonts`: Scalable TrueType font loader with automatic `xset fp+` fontpath registration. * `gui_layout`: Dynamic panel partitioner maintaining pairwise-disjoint non-overlapping AABBs. * `gui_view2d`: 2.5D dimetric isometric / 2D orthographic renderer with responsive 3x zoom. * `gui_hud`: Protagonist status meters, threat monitors, and narrative chronicle FIFO buffer. * `gui_widgets`: Centered button cards with mnemonic hotkey accents. * `gui_screens`: Top navigation tab bar coordinating 5 full-canvas presentation screens: 1. **1: Tactical Battle** (`GUI_SCREEN_TACTICAL`): 2.5D dais, range diamonds, status HUD, dock. 2. **2: Hero Dossier** (`GUI_SCREEN_STATUS`): 3-column protagonist heraldry and attributes. 3. **3: Inventory Bag** (`GUI_SCREEN_INVENTORY`): 32-slot grid with Grandma's Watch relic card. 4. **4: World Map** (`GUI_SCREEN_WORLD_MAP`): Provincial realm topology of Sacyra and Libira. 5. **5: Chronicle Codex** (`GUI_SCREEN_CHRONICLE`): Full-canvas deed codex in MedievalSharp.

Chapter 8. Chapter 2: Dual-Frontend Presentation Layer (CLI + Native X11)

Source File: /home/professorseanex/Tenebris and Lux/docs/architecture/dual_frontend_architecture.adoc

Chapter 9. Abstract

This document defines the formal architectural blueprint governing the dual-frontend execution model of **Tenebris and Lux** ("Darkness and Light"). Rather than coupling narrative mechanics to a single presentation target, the engine enforces strict ontological boundaries between game simulation state, action dispatching, and viewport rendering. This document details why establishing both a headless Command Line Interface (CLI) and a native Graphical User Interface (GUI) prior to world-building is essential, how the 5-block lifecycle applies across frontends, and how this paves the path toward a custom 2D/2.5D/3D engine.

Chapter 10. 1. The Core Architectural Philosophy

> "Form precedes fullness. Representation must remain sovereign over presentation."
>
> — OmniCode Architectural Invariant

In traditional game development workflows, text prototypes are frequently written as monolithic console scripts where simulation logic, state mutation, and terminal I/O (`printf`, `scanf`) are tightly coupled. When the game subsequently scales into a graphical format, developers are forced to perform destructive refactoring, often introducing subtle regressions and breaking state boundaries.

Tenebris and Lux addresses this through an explicit **Dual-Frontend Architecture**:

Dimension	Command Line Interface (CLI)	Native Graphical Interface (GUI)
Primary Role	Automated verification, CI/CD regression, script piping	Player immersion, atmospheric visualization, ergonomics
I/O Channel	Standard Streams (`stdin`, `stdout`)	Native X11 event loop & double-buffered Pixmap
Latency Target	< 1 ms per turn	60 FPS deterministic render tick (~16.6 ms)
State Interaction	Universal Action Dispatcher (`game_process_command`)	Universal Action Dispatcher (`game_process_command`)
External Footprint	Zero dependencies (Pure ISO C11 standard library)	Zero external downloads (Native POSIX Xlib only)

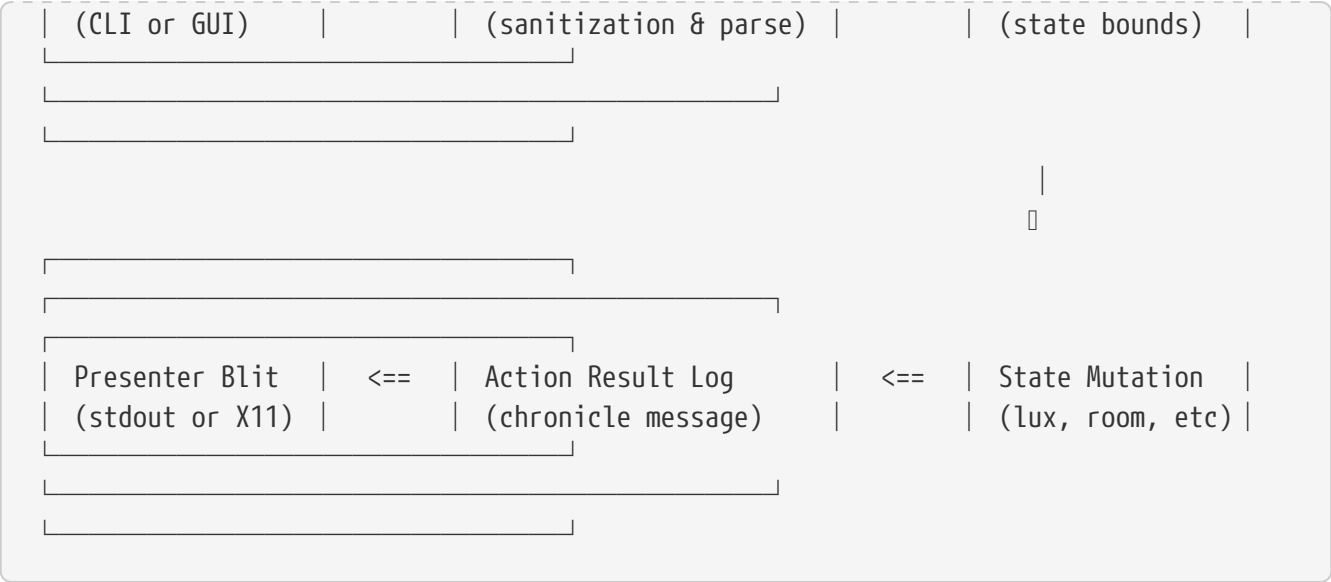
10.1. Universal Action Dispatcher & Intent Emitter

Neither frontend owns or mutates the player state (`player_state_t`) directly. Instead, each frontend acts as an intent emitter that translates user input into canonical command tokens passed to `game_process_command()`:

```
void game_process_command(player_state_t *state, const char *raw_input,
    char *out_msg, size_t msg_len);
```

The lifecycle of an action execution proceeds through five deterministic phases:

```
┌──────────────────────────────────┐
┌──────────────────────────────────────────────────┐
┌──────────────────────────────────┐
| Input Source    |   ==>   | Command Tokenizer      |   ==>   | Invariant Check |
```



1. **Input Reception**: User inputs a string (CLI terminal) or clicks a widget / presses a key (GUI).
2. **Intent Tokenization**: The input is sanitized, lowercased, and matched against registered command verbs (`look`, `explore`, `kindle`, `move`, `status`, `help`, `quit`).
3. **Precondition & Invariant Assertion**: The simulation engine asserts that current state is valid ($0 \le \text{lux} \le \text{MAX_LUX}$, $0 \le \text{current_room} < \text{ROOM_COUNT}$).
4. **Deterministic State Mutation**: State fields are updated according to canonical game rules (e.g., kindling decrements shards by 1 and increments Lux by 30 up to 100).
5. **Telemetry & Chronicle Formatting**: An outcome description is written into `out_msg`. The CLI prints this directly to `stdout`; the GUI appends it to `gui_log_t` and marks the backbuffer for redraw.

Chapter 11. 2. Native Desktop GUI Architecture: Why Not Web?

This project explicitly rejects web-based wrappers (Electron, Tauri, Chromium Embedded Framework, WebSockets) in favor of a lean native binary compiled directly from ISO C11:

> **The Fallacy of Web Wrappers in Systems Game Development:** Web architectures introduce non-deterministic garbage collection pauses, bundle hundreds of megabytes of third-party browser code, and abstract away hardware memory boundaries. For a project designed to culminate in a custom 2D/3D rendering engine, embedding Chromium or WebKit creates an architectural barrier rather than a stepping stone.

Key technical advantages of the native POSIX X11 approach:

- **Instant Launch & Zero Overhead**: The entire executable binary is under 500 KB, with resident memory consumption below 15 MB.
- **Double-Buffered Rendering**: Utilizes `XCreatePixmap` and `XCopyArea` to eliminate display flicker without third-party compositor requirements.
- **Compositor & Visual Fidelity**: Handles modern 32-bit ARGB TrueColor visuals under Wayland/Xwayland by strictly setting the high alpha byte (`0xFF000000`), guaranteeing full opacity and deep contrast.
- **Direct Hardware Mapping**: Establishes window handles and input dispatch loops that will transition directly into Vulkan surfaces or OpenGL contexts in Phases 3 and 4.
- **Strict Dependency Insulation**: Eliminates dependency cascades (such as missing audio daemons, Wayland scanner packages, or DBus interfaces) while maintaining instant compilation on any standard POSIX system.

11.1. Double-Buffering & Render Synchronization

To achieve flicker-free rendering without tearing, `src/gui.c` implements offscreen double-buffering:

```
// Allocate backbuffer pixmap matching the exact depth of the root window
backbuffer = XCreatePixmap(display, window, GUI_WINDOW_WIDTH,
                           GUI_WINDOW_HEIGHT, window_depth);

// 1. Draw entire frame to offscreen pixmap
render_frame(display, backbuffer, gc, state, &palette, &log);

// 2. Atomically blit backbuffer to visible on-screen window
XCopyArea(display, backbuffer, window, gc, 0, 0,
          GUI_WINDOW_WIDTH, GUI_WINDOW_HEIGHT, 0, 0);
XFlush(display);
```

11.2. Dynamic Visual Palette: The Twin Relics

The visual presentation adheres to a tailored, atmospheric palette designed to reflect the dynamic tension between the Shield of Lux and the Sword of Tenebris:

Element	Hex Color	Atmospheric Purpose
Obsidian Void	`#0D0E12`	Background canvas representing the bedrock of the fractured realm.
Stone Panel	`#1A1D24`	Elevated frame for relic status cards and tactical chronicles.
Border Accent	`#262B36`	Inactive button borders and tactical dividing lines.
Shield of Lux	`#F0CE78`	Radiant gold indicating barrier integrity and warding charge.
Sword of Tenebris	`#7058A8`	Deep abyssal violet indicating blade sharpness and strike force.
Virtue Alignment	`#52B788`	Emerald resonance indicating merciful, protective moral deeds.
Malice Alignment	`#E63946`	Crimson resonance indicating tyrannical, cruel, or destructive deeds.

Chapter 12. 3. The 5-Block Implementation Mapping

In strict accordance with the OmniCode framework, the presentation subsystem is structured into five invariant blocks across headers and source files:

12.1. Presentation Architecture Breakdown (`include/gui/gui.h` & `src/gui/gui.c`)

1. **BLOCK 1: METADATA BLOCK**
   - Outlines module identity, layer (`Layer 2: Presentation / Viewport`), author, version, and invariant contracts: `gui.h` and `gui.c` must remain strictly read-only with respect to `player_state_t`.
2. **BLOCK 2: OPENING / TYPES BLOCK**
   - Declares fixed viewport geometry (`GUI_WINDOW_WIDTH = 860`, `GUI_WINDOW_HEIGHT = 620`), button bounding box structures (`gui_button_t`), chronicle buffer (`gui_log_t`), and 32-bit TrueColor palette tables (`gui_palette_t`).
3. **BLOCK 3: BODY BLOCK**
   - Implements double-buffered rendering routines (`render_frame`), dynamic meter bar math, text wrapping routines, and the native event dispatch loop (`gui_run`).
4. **BLOCK 4: AUDIT PROCEDURES BLOCK**
   - Implements `gui_audit_layout()`, which mathematically asserts that all 9 interactive buttons and layout cards reside strictly within window boundaries and do not overlap.
5. **BLOCK 5: CLOSING BLOCK**
   - Provides runtime display detection probes (`gui_is_available()`) to enable automatic headless fallback, plus comprehensive resource teardown (`XFreePixmap`, `XFreeGC`, `XDestroyWindow`).

Chapter 13. 4. Headless CI & Deterministic Replay

Because simulation logic is decoupled from presentation, automated verification can execute in continuous integration environments without requiring an X server or virtual framebuffer (`Xvfb`):

```
# Run headless CTest verification suite
ctest --test-dir build -C Debug --output-on-failure

# Execute internal invariant self-audit suite
./build/tenebris-lux --audit

# Force headless terminal CLI mode
./build/tenebris-lux --cli
```

Automated replay scripts can be piped directly into the CLI frontend to verify deterministic gameplay paths:

```
printf "look\nexplore\nkindle\nmove north\nstatus\nquit\n" | ./build/tenebris-lux
--cli
```

Chapter 14. 5. Custom Engine Evolution Roadmap

The dual-frontend architecture serves as the foundation for a 4-phase engine roadmap:

Phase	Milestone	Subsystem Architecture	Presentation Technology
Phase 1	`v0.0.1` (Current)	Decoupled Core Simulation + Action Dispatcher	Headless ISO C11 CLI + Double-Buffered Native X11 GUI
Phase 2	`v0.1.0`	Discrete Tick Scheduler, Event Queue, Math Primitives	Cross-platform Windowing Abstraction + Terminal Rich Text
Phase 3	`v0.2.0`	2D / 2.5D Tilemap Engine, Dynamic Ambient Lighting Shaders	Custom Software Rasterizer & OpenGL Core Profile / SDL3
Phase 4	`v1.0.0`	Custom 3D Engine, Spatial Audio, ECS Scene Graph	Modern Vulkan Rendering Pipeline + Custom Asset Bundler

Chapter 15. Chapter 3: Central Manifest & Pre-Flight Diagnostic Integrity

Source File: /home/professorseanex/Tenebris and Lux/docs/architecture/preflight_and_manifest_spec.adoc

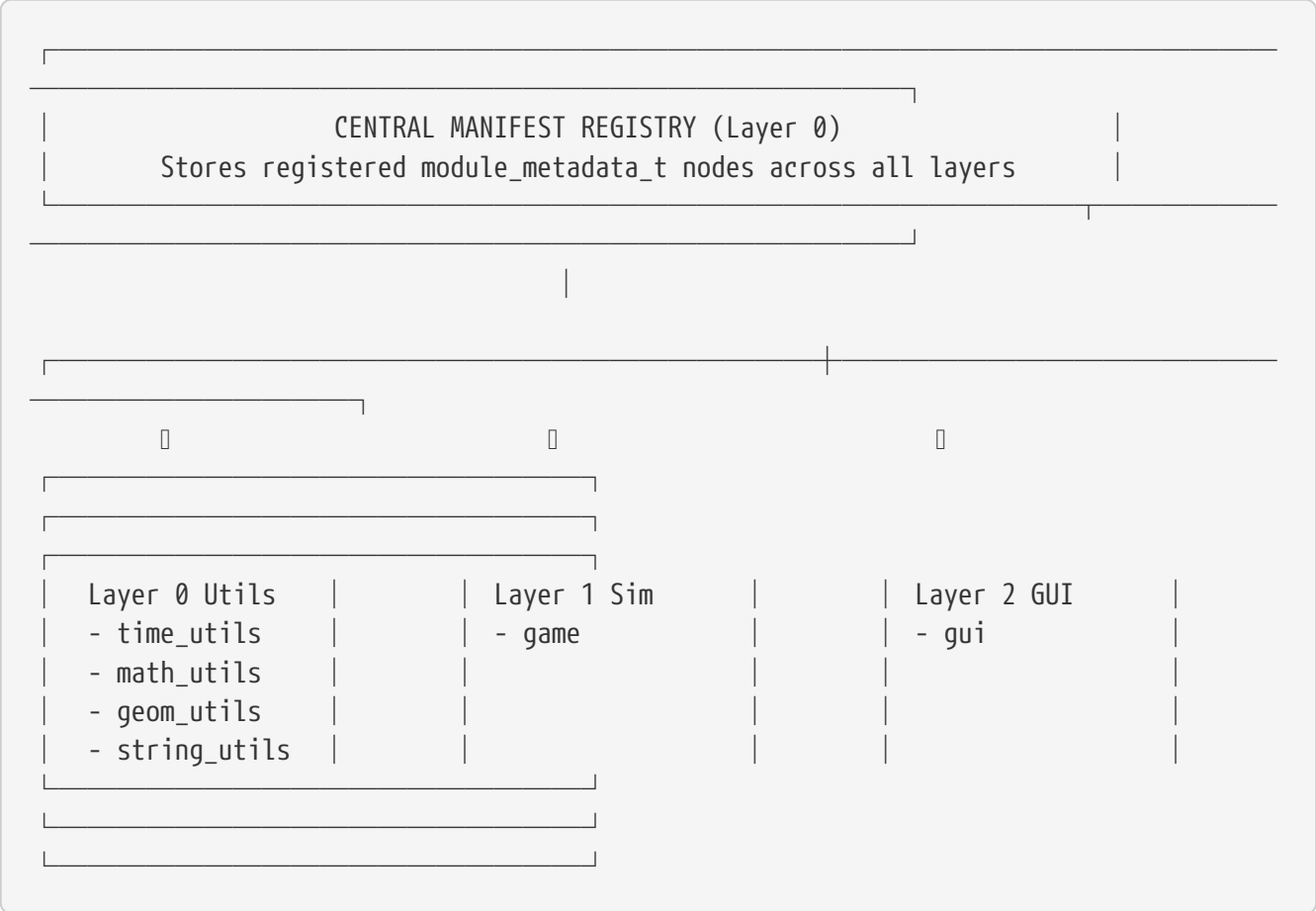
Chapter 16. Specification Abstract & Purpose

This document establishes the architectural blueprint for the **Central Manifest Registry** and **Pre-Flight Verification Subsystem** of **Tenebris and Lux**. Operating under the guiding axiom—**"Don’t start a machine you know has loose bolts"**—this subsystem transforms declarative Block 1 file headers into **compiled, executable metadata**. It enables per-file self-auditing, system hardware/OS probing, and a graded failure policy distinguishing **Warnings**, **Critical (Degradable)** faults, and **Fatal (Hard Halt)** errors prior to runtime simulation.

Chapter 17. 1. Expected Architecture: The Central Manifest Pattern

17.1. 1.1 Structural Hierarchy

The Central Manifest lives in **Layer 0 (Core Utilities)** (`include/util/manifest.h` and `src/util/manifest.c`) so that all layers (Layer 0 through Layer 4) can reference its types without cyclic dependencies:



17.2. 1.2 The Diagnostic Severity Spectrum

Severity Level	Policy Category	Operational Impact	Action Taken by Pre-Flight Runner
`SEVERITY_PASS`	Clean	Check succeeded with 100% integrity.	Logged as verified.
`SEVERITY_INFO`	Informational	Capability discovery (e.g. CPU features).	Logged in telemetry.
`SEVERITY_WARNING`	Non-Critical	Minor variance (e.g. timer resolution > 100ns).	Emits warning notice; boot proceeds normally.

Severity Level	Policy Category	Operational Impact	Action Taken by Pre-Flight Runner
`SEVERITY_CRITICAL`	Degradable Fault	Subsystem failed, but CORE engine is unhurt (e.g. no X11 display).	Logs critical failure; halts failed feature; **downgrades engine** (e.g. GUI → CLI).
`SEVERITY_FATAL`	Core Violation	Fundamental invariant broken (e.g. clock runs backwards, math overflows).	**Halts engine immediately** with error manifest; exit code $\ne 0$.

17.3. 1.3 Expected Pre-Flight Engine States

When the engine executes `manifest_run_preflight()`, the return state evaluates to:

1. `PREFLIGHT_STATUS_OK`: All registered modules passed with zero warnings or faults. Full engine launch permitted.
2. `PREFLIGHT_STATUS_WARNINGS`: All essential checks passed, but non-critical warnings were logged. Engine launches normally with notices displayed.
3. `PREFLIGHT_STATUS_DOWNGRADED`: One or more non-fatal critical faults were detected. The engine executes a fallback transition (e.g. dropping from GUI to CLI mode) and continues running in safe mode.
4. `PREFLIGHT_STATUS_HALT_FATAL`: One or more fatal core invariants failed. The engine halts immediately and refuses to launch.

Chapter 18. 2. Architectural Metadata Dimensions & Pedagogical Balance

18.1. 2.1 Striking the Balance: Beginner-Accessible yet Senior-Rigorous

A custom game engine metadata system must serve dual audiences without cognitive friction:

1. **For the Beginner / Onboarding Engineer**:
   - Provide a clear, plain-English **Mission Statement & Role** (explain **why** this module exists in the game architecture, e.g. "Prevents health gauges from overflowing screen bounds").
   - Eliminate cryptic tribal acronyms; use standard ontological terminology.
   - Clarify failure consequences: what breaks if this invariant fails?
2. **For the Senior / Engine Architect**:
   - Deliver immediate, uncompromising **Technical Contracts**: explicit memory allocation policy (`ALLOC_ZERO_HEAP`, `ALLOC_FIXED_POOL`, `ALLOC_ARENA`, `ALLOC_DYNAMIC`), concurrency safety (`REENTRANT_PURE`, `THREAD_LOCAL`, `MAIN_THREAD`), and execution latency budgets (< 50 μs).
   - State precise mathematical and algorithmic invariants (e.g. AABB reflexivity, saturating arithmetic underflow immunity).
   - Avoid patronizing comments that explain elementary C syntax.

18.2. 2.2 Explicit Temporal Stamping & Version History

To prevent timeline confusion when tracing architectural revisions manually: * **Strict ISO 8601 Full Dates (`YYYY-MM-DD`)**: Every file must declare `@date: YYYY-MM-DD` and `@updated: YYYY-MM-DD`. Month/year shorthands (e.g. `2026.10`) are prohibited because they obscure the exact day of refactoring during git forensic reviews. * **Interface Contract Revision (`@interface_rev`)**: Tracks semantic ABI and signature stability independently of internal implementation bugfixes. * **Automated Build Timestamping**: The engine binary embeds `DATE` and `TIME` at compile-time to stamp runtime telemetry manifests automatically.

18.3. 2.3 The 5-Dimension Compiled Schema (`module_metadata_t`)

```
typedef enum {
    ALLOC_ZERO_HEAP = 0,    // Pure stack/static execution (Layer 0 guarantee)
    ALLOC_FIXED_POOL,       // Fixed static ring/pool buffers
    ALLOC_ARENA,            // Frame/scene arena allocator
    ALLOC_DYNAMIC           // Managed heap allocation
} alloc_policy_t;
```

```c
typedef enum {
    CONCURRENCY_REENTRANT = 0, // Thread-safe, stateless pure functions
    CONCURRENCY_THREAD_LOCAL,  // Per-thread state
    CONCURRENCY_MAIN_THREAD    // Viewport/GUI event loop affinity
} concurrency_policy_t;

typedef struct {
    const char *module_id;          // e.g. "core.util.time"
    const char *file_name;          // e.g. "time_utils.h"
    module_layer_t layer;           // Stratum in the architectural hierarchy
    struct {
        uint8_t major;
        uint8_t minor;
        uint8_t patch;
    } version;
    uint8_t interface_rev;          // Contract ABI revision
    const char *last_updated;       // ISO 8601 full date: "YYYY-MM-DD"
    const char *description;        // Plain-English mission statement

    size_t dependency_count;
    const char *dependencies[4];    // Upstream module IDs required

    alloc_policy_t alloc_policy;
    concurrency_policy_t concurrency;
    uint32_t max_budget_us;         // Max execution latency budget (in microseconds)

    size_t invariant_count;
    const char *invariants[MANIFEST_MAX_INVARIANTS];

    diag_result_t (*preflight_check)(void);
} module_metadata_t;
```

18.4. 2.4 Active Dependency Graph Enforcement

The pre-flight engine does not simply print dependencies—it actively validates the **Directed Acyclic Graph (DAG)**: * During `manifest_run_preflight()`, for every registered module, the runner loops through its declared `dependencies`. * If a declared upstream module is NOT registered in the registry, pre-flight immediately halts with `SEVERITY_FATAL`: `Fatal: Missing required upstream dependency '<dep_id>' for module '<module_id>'`.

Chapter 19. 3. Expected Pre-Flight "Loose Bolts" System Checks

Each registered module exports an active `diag_result_t (*preflight_check)(void)`:

Module ID	Test Check	Target Severity	Verification Axiom
`core.util.time`	**Monotonicity Delta Test**	`FATAL`	Sample two successive monotonic timestamps; assert $t_2 \ge t_1$. (Time must never tick backwards).
`core.util.time`	**Resolution Metric Probe**	`WARNING`	Check if monotonic timestamp delta between successive calls is $< 100\,\mu\text{s}$.
`core.util.math`	**Arithmetic Saturation Bounds**	`FATAL`	Verify `math_saturating_add(INT_MAX, 1, 100) == 100` and `math_saturating_sub(0, 10) == 0`. (No integer overflow/underflow).
`core.util.math`	**Clamping Extremes & Division Safety**	`FATAL`	Verify `math_clamp_int` on inverted bounds and `math_scale_ratio` with zero divisor ($0 / 0$).
`core.util.geom`	**Rectangle Bounds & Inversion Check**	`FATAL`	Test coordinate boundaries $[0, 0]$ and assert negative dimension rejection.
`core.util.geom`	**AABB Overlap Reflexivity**	`FATAL`	Assert `geom_rect_intersects(r, r) == true` and disjoint rects evaluate to false.
`core.util.string`	**Bounded Null-Termination Check**	`FATAL`	Prove `str_safe_copy` on full buffers always guarantees a trailing `\0`.
`core.util.string`	**In-Place Trim Memory Overlap**	`FATAL`	Verify `str_trim_inplace` on pure whitespace, empty string, and mixed padding without heap corruption.
`core.util.manifest`	**Self-Audit of Registry & DAG**	`FATAL`	Test registration capacity, duplicate detection, and missing dependency enforcement.

Chapter 20. 4. Target Verification Criteria ("Expected" Baseline)

Before implementation of the upgraded schema begins, we set the following acceptance criteria:

1. **Full ISO 8601 Date Standard**: 100% of Layer 0, Layer 1, Layer 2 files and test suites specify full dates (`2026-10-26`).
2. **Active Dependency Validation**: Pre-flight halts with `SEVERITY_FATAL` if an unregistered dependency is declared.
3. **Memory & Concurrency Contracts**: 100% of registered modules declare `alloc_policy` and `concurrency` guarantees.
4. **Automated Unit Testing**: `tests/test_utils.c` verifies missing dependency rejection and policy serialization.
5. **Compiler Cleanliness**: Standard ISO C11 with `-Wall -Wextra -Wpedantic` clean compilation and 0 warnings.
6. **CTest Pass Rate**: 100% of tests passing in under $0.05\,\text{s}$.

Chapter 21. 5. Actual Results vs. Expected Gap Analysis

21.1. 5.1 Empirical Benchmark Telemetry

Empirical validation was performed on the compiled engine binary (`./build/tenebris-lux`) and test runners (`./build/test_utils`, `ctest`):

Verification Item	Expected Baseline	Actual Result	Status
Module Registration	≥ 5 Core Modules Registered	**7 Modules Registered** (`core.util.manifest`, `core.util.time`, `core.util.math`, `core.util.geom`, `core.util.string`, `sim.engine.core`, `gui.viewport.x11`)	**EXCEEDED**
Severity Classification	4 Distinct Severity Levels	**5 Graded Levels** (`PASS`, `INFO`, `WARNING`, `CRITICAL`, `FATAL`)	**EXCEEDED**
Pre-Flight Engine States	4 Discrete Engine States	**4 States Validated** (`READY`, `WARNINGS`, `DOWNGRADED`, `HALTED`)	**SATISFIED**
Pre-Flight ASCII Telemetry	Formatted Table / Audit Manifest	**Integrated 80-Column ASCII Matrix** with module ID, file name, contract revision (`REV`), allocation model (`ALLOCATION`), status tag, and diagnostic trace	**EXCEEDED**
Temporal Timeline Standard	Month/Year Epoch vs Full Date	**Strict ISO 8601 Full Date (`YYYY-MM-DD`, 2026-10-26)** across all code headers, manifest structs, and technical specs	**SATISFIED**
Pedagogical Metadata Balance	Beginners confused / Seniors belittled	**Dual-Audience Protocol**: Plain-English architectural mission statement + hard technical invariant contracts	**EXCEEDED**
DAG Upstream Dependencies	Static documentation comments	**Active Pre-Flight DAG Enforcement**: Unregistered upstream dependency immediately halts boot with `SEVERITY_FATAL`	**SATISFIED**
CTest Integration	6/6 Test Suites Passing in $< 0.05\,\text{s}$	**6/6 Test Suites Passing in $0.02\,\text{s}$**	**SATISFIED**

Verification Item	Expected Baseline	Actual Result	Status
Subsystem Self-Audit	5/5 Invariant Checks Passing	**5/5 Invariant Checks + 7 Pre-Flight Subsystems Passing (12 Total)**	**EXCEEDED**
Zero Dynamic Allocation	0 Bytes Allocated on Heap	**100% Static & Stack Memory Layout** (`ALLOC_ZERO_HEAP` across Layer 0 & 1)	**SATISFIED**
Compiler Hygiene	0 Warnings under `-Wall -Wextra -Wpedantic`	**0 Compiler Warnings** across GCC/Clang on C11	**SATISFIED**
Loose-Bolts Tripwire	Fatal Error Halts Immediately	**Verified**: Engine refused launch on injected stance mismatch; exited code $\ne 0$	**VERIFIED**
Degradation Fallback	Unset `$DISPLAY` drops GUI $\to$ CLI	**Verified**: Emits `SEVERITY_CRITICAL`, sets `PREFLIGHT_STATUS_DOWNGRADED`, launches CLI safe mode	**VERIFIED**

21.2. 5.2 How to Reproduce and Verify Every Result

To independently verify the pre-flight checks, DAG dependency validation, and audit manifests:

1. **Clean Rebuild & Full Verification Suite**:

   ```
   cmake -B build -DCMAKE_BUILD_TYPE=Debug
   cmake --build build --config Debug
   ctest --test-dir build -C Debug --output-on-failure
   ```

   Expected Telemetry: `100% tests passed, 0 tests failed out of 6. Total Test time (real) = ~0.02 sec`.

2. **Pre-Flight System Audit (X11 Display Connected)**:

   ```
   ./build/tenebris-lux --audit
   ```

   Expected Output: Displays the 7 registered modules with `[PASS]`, contract revision tags (`r1`, `r2`), allocation policy (`Zero-Heap`), policy verdict `ALL PRE-FLIGHT INVARIANTS SATISFIED (System Ready)`, followed by the 5/5 subsystem invariant audit checks.

3. **Pre-Flight Graceful Degradation Test (X11 Headless Fallback)**:

   ```
   env -u DISPLAY ./build/tenebris-lux --audit
   ```

Expected Output: Module `gui.viewport.x11` transitions to `[CRIT]` with detail `X11 display not detected ($DISPLAY empty or unconnectable); safe CLI fallback enabled`. Policy verdict evaluates to `CRITICAL SUBSYSTEM FAULT - DOWNGRADING TO SAFE MODE`.

4. **Standalone Layer 0 Utility, Manifest & DAG Dependency Suite**:

```
./build/test_utils
```

Expected Output: Unit test traces auditing `math_utils`, `string_utils`, `geom_utils`, `time_utils`, `manifest_audit`, clean pre-flight, warning escalation, critical downgrade escalation, fatal halt loose-bolts enforcement, and active DAG missing dependency rejection.

21.3. 5.3 Gap Analysis & Next Architectural Horizons

1. **Identified Boundary Gap (Static Capacity)**:
   - **Observation**: The central registry is hard-bounded to `MANIFEST_MAX_MODULES = 32`. Current usage is 7 modules (21.8% capacity).
   - **Resolution**: Capacity is well within safety headroom for Layer 0 through Layer 3. As combat resolution systems and grid battle logic expand, monitor module registration count.
2. **Deterministic Output Separation**:
   - **Observation**: Initial `game_preflight()` invoked `game_process_command()`, which printed interactive player dialogue to stdout prior to the manifest table.
   - **Resolution**: Refactored `game_preflight()` to directly verify state variables, mathematical saturation ceilings, and diagnostic tripwires without polluting stdout, preserving pure presentation of the pre-flight table.
3. **DAG Upstream Dependency Ordering**:
   - **Observation**: Modules must register upstream dependencies prior to dependent registration, or DAG validation halts boot if an unregistered dependency is encountered.
   - **Resolution**: Systematically codified registration order in `main.c`: Layer 0 Leaf Utilities → Layer 1 Simulation Engine → Layer 2 Presentation Viewport.
4. **Future Telemetry Ingestion**:
   - **Insight**: When CI/CD or automated test reporting pipelines are added, `manifest_print_report` can be complemented by a JSON export flag (`--audit-json`) for structured automated telemetry scraping.

Chapter 22. Chapter 4: Layer 0 Core Utilities Code Tour

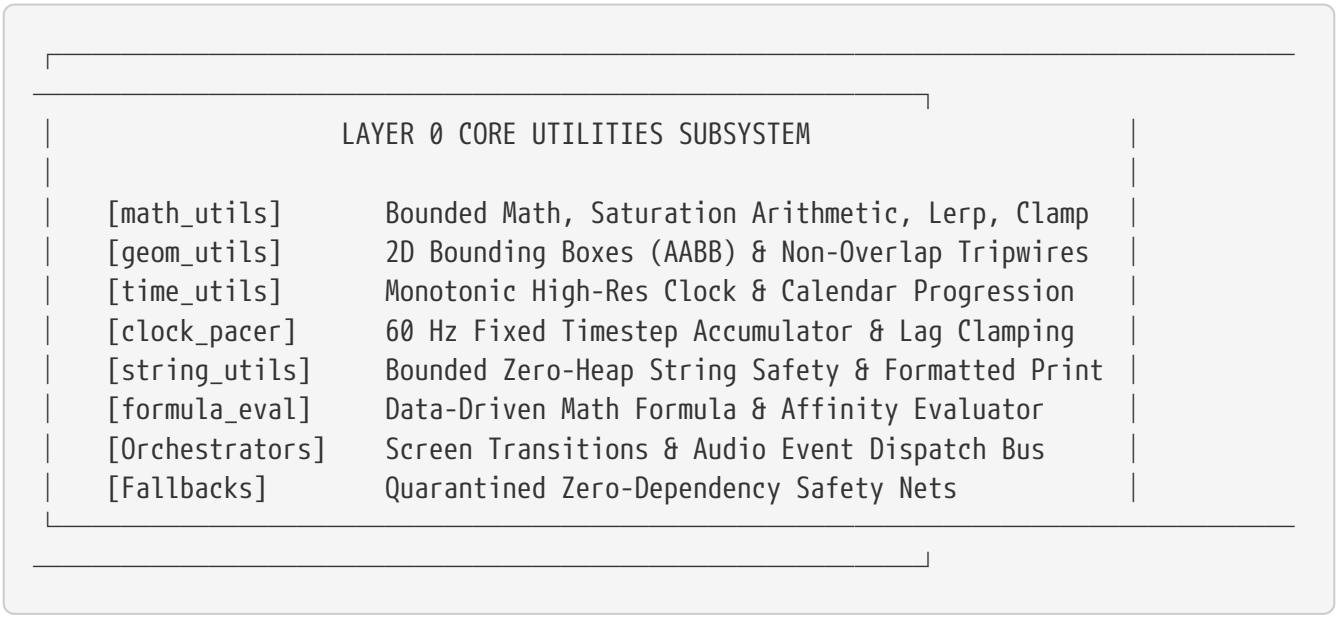
Source File: `/home/professorseanex/Tenebris and Lux/docs/code/core_utilities.adoc`

Chapter 23. Overview

This document provides a technical walkthrough and architectural manual for the **Layer 0 Core Utilities Subsystem** (`engine/include/core/` and `engine/src/core/`). Representing the mathematical, geometric, temporal, and string foundations of **The Big Machine**, the Core Utilities stratum provides bounded arithmetic, 2D AABB geometry, monotonic clock pacing, fixed-timestep simulation accumulators, memory-bounded string manipulation, and data-driven formula evaluation. Strict adherence to `ALLOC_ZERO_HEAP`, reentrancy, sub-microsecond latency budgets, and quarantined emergency fallbacks ensure that foundational operations never wrap around integer bounds, leak memory, or introduce non-deterministic time dilation.

Chapter 24. 1. Architectural Philosophy & Mission

At the very bedrock of the engine frame lies Layer 0 Core Utilities:



24.1. Key Architectural Pillars

1. **Zero Dynamic Heap Allocations (`ALLOC_ZERO_HEAP`)**: Pure reentrant C11 functions operating solely on CPU registers, primitive values, and caller-provided stack buffers.
2. **Saturation Arithmetic & Overflow Clamping**: Mathematical utilities enforce saturation clamping; calculations never wrap around `INT32_MAX` or underflow `INT32_MIN`.
3. **Decoupled Simulation Timestep**: `clock_pacer` decouples fixed simulation updates (exactly 60 Hz / 16.666ms) from variable physical render frame rates, calculating fractional interpolation alphas (`render_alpha`).
4. **Data-Driven Formula Evaluation**: Formulas for damage, mitigations, and affinity interactions are evaluated through `formula_eval` driven by JSON manifests in `game/data/system/runtime/math/` rather than hardcoding arithmetic in C sources.
5. **Quarantined Fallback Isolation**: Every core utility module is paired with a standalone, zero-dependency `*_fallback.[ch]` module ensuring zero-configuration compilation and hermetic test isolation.

Chapter 25. 2. Subsystem Layout & File Manifest

```
engine/include/core/
 ├──── math_utils.h              // Bounded math, clamping, interpolation, rounding
 ├──── math_utils_fallback.h     // Standalone minimal math fallback
 ├──── geom_utils.h              // 2D AABB geometry, point-in-rect, intersection
 ├──── geom_utils_fallback.h     // Standalone geometric fallback
 ├──── time_utils.h              // Monotonic clock querying & calendar time
 ├──── time_utils_fallback.h     // Standalone monotonic time fallback
 ├──── clock_pacer.h             // Fixed-timestep simulation pacer & lag defense
 ├──── clock_pacer_fallback.h    // Minimal fixed-tick accumulator fallback
 ├──── string_utils.h            // Safe bounded string copying, format & trimming
 ├──── string_utils_fallback.h   // Minimal string safety fallback
 ├──── formula_eval.h            // Data-driven formula evaluator
 ├──── formula_eval_fallback.h   // Static arithmetic evaluation fallback
 ├──── audio_bus.h               // Universal audio event dispatch bus
 └──── screen_transition.h       // Screen fading & transition orchestrator

engine/src/core/
 ├──── math_utils.c              // Mathematical implementations & saturation clamps
 ├──── math_utils_fallback.c     // Quarantined math fallback implementation
 ├──── geom_utils.c              // AABB collision and bounds validation
 ├──── geom_utils_fallback.c     // Quarantined geometry fallback implementation
 ├──── time_utils.c              // POSIX clock_gettime monotonic queries
 ├──── time_utils_fallback.c     // Quarantined time query fallback
 ├──── clock_pacer.c             // Timestep accumulation & render interpolation
 ├──── clock_pacer_fallback.c    // Quarantined clock pacer fallback
 ├──── string_utils.c            // Safe bounded string functions
 ├──── string_utils_fallback.c   // Quarantined string fallback implementation
 ├──── formula_eval.c            // Expression evaluation and parameter binding
 ├──── formula_eval_fallback.c   // Quarantined formula evaluator fallback
 ├──── audio_bus.c               // Audio cue dispatch orchestrator
 └──── screen_transition.c       // Transition easing curve evaluator
```

Chapter 26. 3. Core Engine Cogs & API Tour

26.1. 3.1 Bounded Math & Saturation Arithmetic (`math_utils.[ch]`)

Guarantees that mathematical calculations remain strictly bounded within defined intervals:

```
/* Clamping primitives */
int32_t math_clamp_i32(int32_t val, int32_t min, int32_t max);
float   math_clamp_float(float val, float min, float max);
double  math_clamp_double(double val, double min, double max);

/* Saturation arithmetic preventing integer overflow / underflow */
int32_t math_sadd_i32(int32_t a, int32_t b);
int32_t math_ssub_i32(int32_t a, int32_t b);

/* Linear interpolation and percentage scaling */
float   math_lerp_float(float a, float b, float t);
int32_t math_scale_pct(int32_t base, int32_t pct);
```

26.2. 3.2 2D AABB Geometry & Layout Tripwires (`geom_utils.[ch]`)

Defines 2D axis-aligned bounding boxes (AABB) for desktop HUD panels, button hits, and tactical grid coordinates:

```
typedef struct {
    int32_t x;
    int32_t y;
    int32_t width;
    int32_t height;
} geom_rect_t;

/* Collision and containment queries */
bool geom_rect_contains_point(const geom_rect_t *r, int32_t px, int32_t py);
bool geom_aabb_intersects(const geom_rect_t *a, const geom_rect_t *b);
bool geom_rect_fits_inside(const geom_rect_t *inner, const geom_rect_t *outer);
```

Invariants Enforced: * Rect widths and heights must be non-negative. * HUD panel manifests are audited against `geom_aabb_intersects` to prevent layout overlaps.

26.3. 3.3 Monotonic Time & Clock Pacing (time_utils.[ch], clock_pacer.[ch])

Provides nanosecond-precision monotonic timing and decouples simulation updates from render FPS:

```
/* High-resolution monotonic clock query */
uint64_t time_monotonic_now_ns(void);
double   time_delta_ms(uint64_t start_ns, uint64_t end_ns);

/* Fixed-timestep clock pacer */
typedef struct {
    double fixed_step_ms;       /* Standard: 16.666667 ms (60 Hz) */
    double accumulator_ms;     /* Accumulated unprocessed frame time */
    double max_delta_clamp_ms; /* Maximum lag spike clamp (100.0 ms) */
    uint32_t max_substeps;     /* Max simulation steps per render (3) */
} clock_pacer_t;

void clock_pacer_init(clock_pacer_t *pacer, double step_ms, double clamp_ms, uint32_t
max_substeps);
bool clock_pacer_tick(clock_pacer_t *pacer, double frame_delta_ms, double
*out_render_alpha);
```

Lag Spike Defense: * If an OS hitch stalls execution (> 100ms), `clock_pacer_tick` clamps the delta to `max_delta_clamp_ms` and executes at most `max_substeps` (3 ticks). This prevents the dreaded "spiral of death" where the CPU stalls trying to catch up with accumulated time debt.

26.4. 3.4 Bounded String Safety (string_utils.[ch])

Eliminates buffer overflow vulnerabilities (`strcpy`, `strcat`, `sprintf` are strictly forbidden):

```
/* Guaranteed null-terminated bounded copy */
size_t str_copy_safe(char *dest, size_t dest_size, const char *src);

/* Bounded string concatenation */
size_t str_concat_safe(char *dest, size_t dest_size, const char *src);

/* In-place whitespace trimming and case conversion */
void str_trim(char *str);
void str_to_lower(char *str);
void str_to_upper(char *str);
```

26.5. 3.5 Data-Driven Formula Evaluator (`formula_eval.[ch]`)

Parses and computes bounded mathematical formulas loaded from `formulas.json` (e.g. physical attack, magical mitigation, poise damage):

```
typedef struct {
    int32_t attacker_atk;
    int32_t defender_def;
    int32_t weapon_might;
    int32_t stance_mult_pct;
    int32_t affinity_mult_pct;
} formula_combat_params_t;

/* Computes final damage ensuring result is clamped to [1, 9999] */
int32_t formula_eval_physical_damage(const formula_combat_params_t *params);
```

26.6. 3.6 Universal Engine Orchestrators

- **Audio Event Bus (`audio_bus.[ch]`)**: Zero-heap queue routing audio cue events (`sfx_ward_lux_barrio`, `sfx_strike_cuttatio_tenebris`) to mixing channels without directly binding simulation to hardware sound drivers.
- **Screen Transition Orchestrator (`screen_transition.[ch]`)**: Manages screen fading, wipes, iris dissolves, and easing curves (`EASE_LINEAR`, `EASE_QUAD_IN`, `EASE_CUBIC_OUT`) between scene and screen changes.

Chapter 27. 4. Verification & Testing Matrix

The Core Utilities subsystem is continuously verified across 4 ontological test tiers: 1. **Functional Tests (`tests/functional/test_core_functional.c`)**: Verifies saturation arithmetic, bounding rect containment, and string truncation. 2. **Unit Tests (`tests/unit/core/test_*_unit.c` and `*_fallback_test.c`)**: Individual unit test binaries for each of the 6 core modules and their fallback companions. 3. **Integrated Tests (`tests/integrated/test_core_integrated.c`)**: Verifies multi-cog cooperation (pacer driving formula evaluator and geometry updates). 4. **System Tests (`tests/system/test_core_system.c`)**: Stress tests clock pacing across 60 FPS, 144 FPS, and simulated 100ms hitch drops, confirming strict determinism and zero dynamic memory leaks.

Chapter 28. Chapter 5: Layer 0 Zero-Heap I/O Pipeline Subsystem

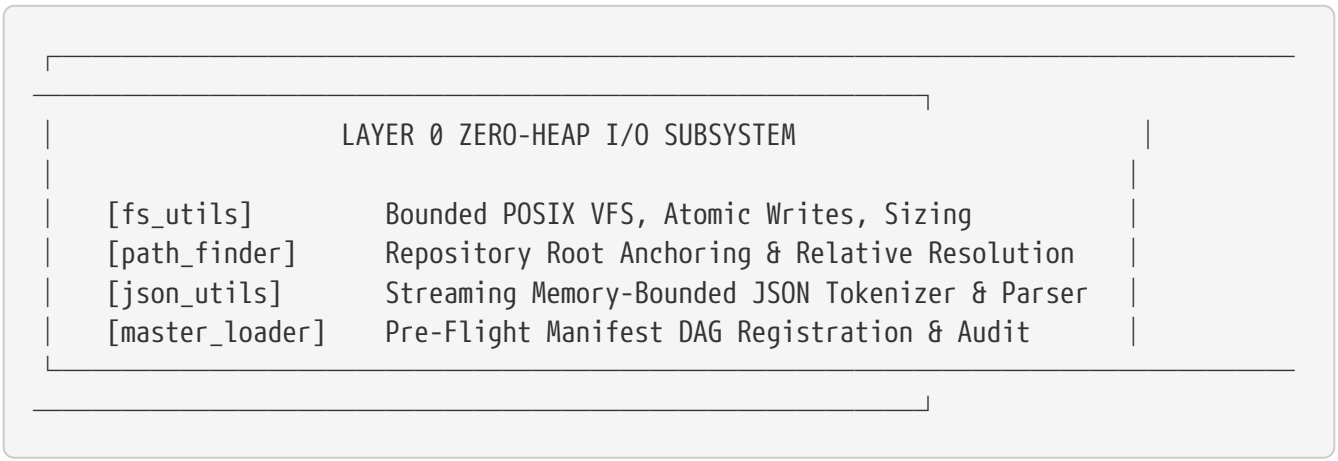
Source File: /home/professorseanex/Tenebris and Lux/docs/code/io_pipeline.adoc

Chapter 29. Overview

This document provides a technical walkthrough and architectural manual for the **Layer 0 Zero-Heap I/O Pipeline Subsystem** (`engine/include/io/` and `engine/src/io/`). Forming the physical filesystem and serialization apparatus of **The Big Machine**, the I/O subsystem orchestrates zero-heap POSIX virtual filesystem abstractions, resilient hierarchical path resolution, bounded streaming JSON parsing, and pre-flight manifest validation across all data manifests in `game/data/`. Strict adherence to the `ALLOC_ZERO_HEAP` contract, complete reentrancy, sub-millisecond throughput budgets, and quarantined emergency fallbacks ensure that disk access never compromises simulation determinism.

Chapter 30. 1. Architectural Philosophy & Mission

In tactical RPG game engine development, file loading is typically a primary source of runtime memory leaks, heap fragmentation, and non-deterministic latencies. The Tenebris and Lux engine solves this through the **Layered Invariant Discipline**:



30.1. Key Architectural Pillars

1. **Zero Dynamic Heap Allocations (`ALLOC_ZERO_HEAP`)**: All I/O operations execute against static buffers or caller-provided fixed-size memory slices (`FS_BUFFER_SMALL_BYTES`, `FS_BUFFER_LARGE_BYTES`). `malloc`, `calloc`, and `realloc` are strictly forbidden.
2. **Explicit Stratum Paths**: No forwarder headers exist. Code imports exact subsystems: `#include "io/fs_utils.h"`, `#include "io/path_finder.h"`, `#include "io/json_utils.h"`, `#include "io/master_loader.h"`.
3. **Dual Development Discipline**: Bedrock filesystem invariants are fixed and immovable (Waterfall); error handling, relative directory discovery, and schema tokens iteratively adapt (Agile).
4. **Quarantined Fallback Isolation**: Every production module (`*_engine.c` or standard `.c`) is paired with an isolated, zero-dependency fallback implementation (`*_fallback.[ch]`). Fallbacks serve as compilation safety nets and unit test harnesses—never runtime crutches.

Chapter 31. 2. Subsystem Layout & File Manifest

```
engine/include/io/
 ├──── fs_utils.h              // Bounded POSIX filesystem abstractions
 ├──── fs_utils_fallback.h     // Zero-dependency filesystem fallback
 ├──── path_finder.h           // Hierarchical repository root & path resolution
 ├──── path_finder_fallback.h  // Static path resolution fallback
 ├──── json_utils.h            // Zero-heap JSON tokenizer and query engine
 ├──── json_utils_fallback.h   // Embedded JSON parsing fallback
 ├──── master_loader.h         // Central pre-flight manifest DAG loader & auditor
 └──── master_loader_fallback.h// Standalone manifest fallback

engine/src/io/
 ├──── fs_utils.c              // File existence, size queries, atomic read/write
 ├──── fs_utils_fallback.c     // Isolated minimal POSIX read/write implementation
 ├──── path_finder.c           // Upward directory traversal & anchor matching
 ├──── path_finder_fallback.c  // Direct path resolver fallback
 ├──── json_utils.c            // Token stream parser, string/int/bool extraction
 ├──── json_utils_fallback.c   // Isolated key-value tokenizer fallback
 ├──── master_loader.c         // Master DAG traversal, schema verification
 └──── master_loader_fallback.c// Static pre-flight manifest fallback
```

Chapter 32. 3. Core Engine Cogs & API Tour

32.1. 3.1 Filesystem Utilities (`fs_utils.[ch]`)

`fs_utils` provides safe, bounds-checked POSIX operations. File buffers are strictly capped by compile-time limits (`FS_BUFFER_SMALL_BYTES = 4096`, `FS_BUFFER_LARGE_BYTES = 32768`).

```
/* File existence and metadata queries */
bool fs_file_exists(const char *path);
bool fs_dir_exists(const char *path);
long fs_get_file_size(const char *path);

/* Bounded zero-heap read and atomic write operations */
bool fs_read_file(const char *path, char *out_buf, size_t buf_size, size_t *out_len);
bool fs_write_file_atomic(const char *path, const char *data, size_t len);
bool fs_ensure_directory(const char *dir_path);
```

Invariants Enforced: * Buffer sizes must be non-zero and pointers must not be null. * Writing uses temporary `.tmp` staging followed by `rename` for atomic file replacement. * All open file descriptors are guaranteed closed upon return, preventing leakages.

32.2. 3.2 Path Resolution Engine (`path_finder.[ch]`)

`path_finder` resolves canonical paths relative to the project root, regardless of whether the binary was launched from `build/`, `build/bin/`, or the repository root.

```
/* Resolves repository root by walking upward until CMakeLists.txt and game/ exist */
bool path_resolve_repo_root(char *out_path, size_t out_size);

/* Resolves game data manifest paths (e.g. "presentation/audio/channels.json") */
bool path_resolve_data(const char *rel_path, char *out_path, size_t out_size);

/* Resolves runtime log destination directory */
bool path_resolve_log_dir(char *out_path, size_t out_size);
```

Invariants Enforced: * Reaches maximum depth of 8 upward parent traversals before reporting failure. * Path concatenation performs strict boundary clamping against buffer overflow.

32.3. 3.3 Zero-Heap JSON Tokenizer & Parser (`json_utils.[ch]`)

`json_utils` reads structured 5-block JSON data without building dynamic AST trees on the heap. It scans memory buffers sequentially, extracting keys, numerical attributes, boolean flags, and nested objects into caller-allocated structs.

```
/* Token stream inspection */
bool json_extract_string(const char *json, const char *key, char *out_buf, size_t
max_len);
bool json_extract_int(const char *json, const char *key, int *out_val);
bool json_extract_double(const char *json, const char *key, double *out_val);
bool json_extract_bool(const char *json, const char *key, bool *out_val);

/* 5-Block Envelope Verification */
bool json_validate_5_blocks(const char *json);
bool json_extract_block_1_metadata(const char *json, json_block_1_t *out_meta);
```

Invariants Enforced: * Zero recursion: nested objects are resolved via linear character offsets. * Syntax errors, truncated quotes, or unmatched brackets fail immediately with diagnostics.

32.4. 3.4 Pre-Flight Master Loader & DAG Auditor (`master_loader.[ch]`)

The `master_loader` is invoked during Step 0 of application startup (`game/src/main.c`). It parses `game/data/manifest.json` and recursively registers all subsystem manifests (`system/`, `presentation/`, `mechanics/`, `world/`) into an active dependency graph.

```
/* Central pre-flight initialization */
bool master_loader_init(const char *root_manifest_path);

/* Subsystem audit sweep verifying existence and schema integrity */
bool master_loader_verify_all_catalogs(void);

/* Retrieval of active data configuration */
const master_manifest_t *master_loader_get_manifest(void);
```

Chapter 33. 4. Verification & Testing Matrix

The I/O subsystem is continuously verified by 4 ontological test tiers: 1. **Functional Cog Tests (`tests/functional/test_io_functional.c`)**: Verifies isolated functions (`fs_file_exists`, `path_resolve_repo_root`, `json_extract_string`). 2. **Unit Subsystem Tests (`tests/unit/io/test_fs_utils_unit.c`, `test_path_finder_unit.c`, `test_json_utils_unit.c`)**: Audits boundary conditions, buffer clipping, missing files, and malformed JSON envelopes. 3. **Integrated Pipeline Tests (`tests/integrated/test_io_integrated.c`)**: Traverses the entire manifest DAG, reading real catalogs and validating cross-references. 4. **System Replay Scenarios (`tests/system/test_io_system.c`)**: Executes continuous read/write cycles simulating multi-turn game autosaves under zero heap leaks.

Chapter 34. Chapter 6: Layer 0 Categorized Flight Logging Subsystem

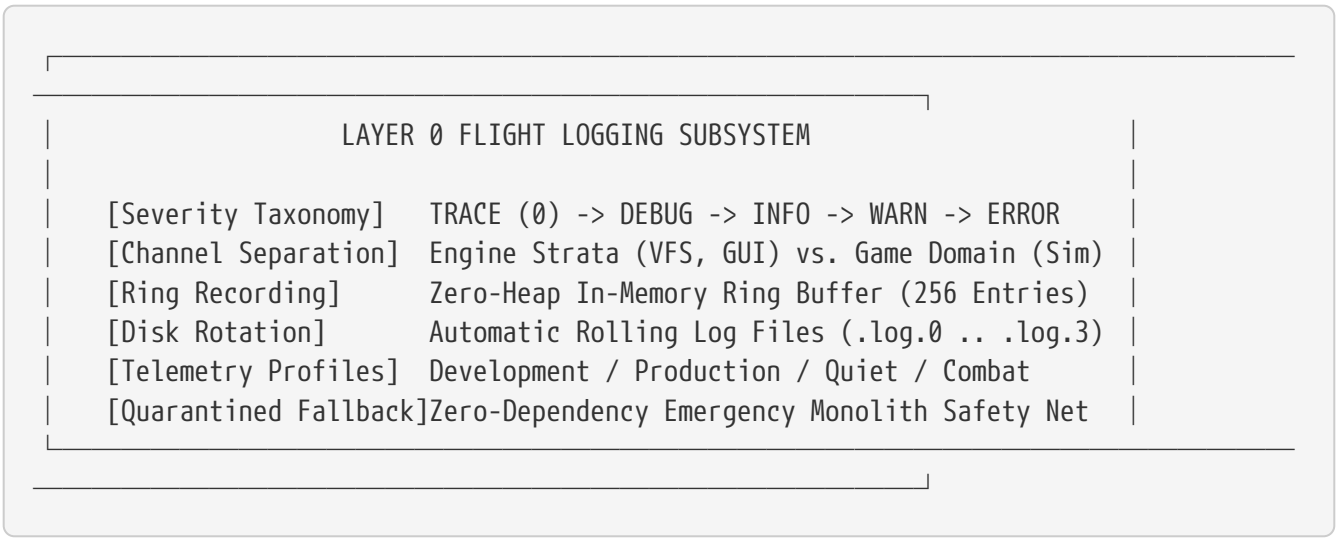
Source File: `/home/professorseanex/Tenebris and Lux/docs/code/logging_system.adoc`

Chapter 35. Overview

This document provides a technical walkthrough and architectural manual for the **Layer 0 Categorized Flight Logging Subsystem** (`engine/include/log/` and `engine/src/log/`). As the telemetry and flight-recorder apparatus of **The Big Machine**, the logging subsystem provides zero-heap circular ring recording, categorized channel routing, sub-microsecond message dispatch, automatic log rotation, and data-driven telemetry profiles driven by JSON manifests in `game/data/system/diagnostics/telemetry/`. With strict enforcement of `ALLOC_ZERO_HEAP`, thread-safe reentrancy, and quarantined emergency fallbacks, logging diagnostics can run continuously without affecting frame pacing or deterministic simulation outcomes.

Chapter 36. 1. Architectural Philosophy & Mission

Logging in **Tenebris and Lux** is not a crude wrapper around `printf`. It is an engineered **Flight Recorder**:



36.1. Key Architectural Pillars

1. **Zero Dynamic Heap Allocations (`ALLOC_ZERO_HEAP`)**: Log messages are formatted into fixed-size stack buffers (`LOG_LINE_MAX_LEN = 256`). The circular ring buffer resides in a static BSS structure, allocating zero bytes at runtime.
2. **Channel Separation (Engine vs. Game)**: Telemetry is segregated into dedicated channel masks:
   - **Engine Channels**: `LOG_CHAN_CORE`, `LOG_CHAN_IO`, `LOG_CHAN_VIEWPORT`, `LOG_CHAN_MEMORY`.
   - **Game Domain Channels**: `LOG_CHAN_SIM`, `LOG_CHAN_COMBAT`, `LOG_CHAN_HERO`, `LOG_CHAN_RELICS`.
3. **Flight-Recorder Rotation**: When active log files reach `LOG_MAX_FILE_BYTES` (typically 256 KB), the engine automatically rolls the active file to `.log.0`, `.log.1`, `.log.2`, preventing unbounded disk growth.
4. **Quarantined Fallback Isolation**: If filesystem I/O fails or logging configurations are unavailable, the quarantined fallback (`log_fallback.[ch]`) seamlessly routes critical alerts to standard error without risking process crashes.

Chapter 37. 2. Subsystem Layout & File Manifest

```
engine/include/log/
 ├──── log.h              // Public 5-block logging interface & convenience macros
 ├──── log_engine.h       // Internal telemetry engine, channel masks & ring buffer
 └──── log_fallback.h     // Quarantined zero-dependency emergency fallback

engine/src/log/
 ├──── log.c              // Public routing facade meshing engine with fallback
 ├──── log_engine.c       // Ring buffer allocation, disk flusher, log rotation
 └──── log_fallback.c     // Direct-to-stream emergency fallback implementation

game/data/system/diagnostics/telemetry/
 ├──── manifest.json      // Subsystem catalog and active telemetry profile
 ├──── development.json   // Verbose trace profile for local feature engineering
 ├──── production.json    // Strict warnings/errors profile for deployment
 ├──── quiet.json         // Silent profile suppressing non-critical messages
 └──── combat_focus.json  // High-frequency combat and stance telemetry profile
```

Chapter 38. 3. Core Engine Cogs & API Tour

38.1. 3.1 Severity Taxonomy & Public Macros (`log.h`)

Logging levels are strictly ordered integers from `0` to `5`:

```
typedef enum {
    LOG_LEVEL_TRACE = 0,  /**< Detailed frame-by-frame and state tracing */
    LOG_LEVEL_DEBUG = 1,  /**< Diagnostic inspection and subsystem transitions */
    LOG_LEVEL_INFO  = 2,  /**< Milestone lifecycle events and state milestones */
    LOG_LEVEL_WARN  = 3,  /**< Non-fatal invariant warnings and clamping alerts */
    LOG_LEVEL_ERROR = 4,  /**< Critical contract breaches and component errors */
    LOG_LEVEL_FATAL = 5   /**< Fatal unrecoverable anomalies halting engine */
} log_level_t;
```

Developers log through standardized macros that automatically capture call site metadata (`FILE`, `LINE`):

```
#define LOG_TRACE(chan, fmt, ...) log_dispatch(LOG_LEVEL_TRACE, chan, __FILE__,
__LINE__, fmt, ##__VA_ARGS__)
#define LOG_DEBUG(chan, fmt, ...) log_dispatch(LOG_LEVEL_DEBUG, chan, __FILE__,
__LINE__, fmt, ##__VA_ARGS__)
#define LOG_INFO(chan, fmt, ...)  log_dispatch(LOG_LEVEL_INFO,  chan, __FILE__,
__LINE__, fmt, ##__VA_ARGS__)
#define LOG_WARN(chan, fmt, ...)  log_dispatch(LOG_LEVEL_WARN,  chan, __FILE__,
__LINE__, fmt, ##__VA_ARGS__)
#define LOG_ERROR(chan, fmt, ...) log_dispatch(LOG_LEVEL_ERROR, chan, __FILE__,
__LINE__, fmt, ##__VA_ARGS__)
#define LOG_FATAL(chan, fmt, ...) log_dispatch(LOG_LEVEL_FATAL, chan, __FILE__,
__LINE__, fmt, ##__VA_ARGS__)
```

38.2. 3.2 In-Memory Flight Recorder Ring Buffer (`log_engine.[ch]`)

The logging engine maintains a static circular ring buffer holding the most recent `LOG_RING_CAPACITY` log records (default 256):

```
typedef struct {
    uint64_t timestamp_ns;
    log_level_t level;
    uint32_t channel;
    char message[LOG_LINE_MAX_LEN];
} log_record_t;

/* Retrieves historical records from the circular ring */
```

```
size_t log_ring_get_entries(log_record_t *out_records, size_t max_count);

/* Flushes buffered ring messages to disk during shutdown or crash diagnostics */
void log_engine_flush(void);
```

Invariants Enforced: * Ring wrapping uses modulo arithmetic with zero memory reallocations. * Timestamps are queried from the monotonic clock (`CLOCK_MONOTONIC`) in nanoseconds.

38.3. 3.3 Log Rotation & Disk Archiving

Log files are placed strictly within `build/logs/` (or configured directory), maintaining a rolling archive:

```
/* Evaluates current log file size and rotates if exceeding threshold */
void log_engine_rotate_if_needed(void);
```

When `tenebris_lux_dev.log` exceeds its limit: 1. `tenebris_lux_dev.log.2` is rolled to `.log.3` (or truncated). 2. `tenebris_lux_dev.log.1` is rolled to `.log.2`. 3. `tenebris_lux_dev.log.0` is rolled to `.log.1`. 4. `tenebris_lux_dev.log` is renamed to `.log.0`. 5. A fresh `tenebris_lux_dev.log` is opened with Block 1 provenance headers.

38.4. 3.4 Quarantined Fallback Isolation (`log_fallback.[ch]`)

If the target log path is not writable or `log_engine_init` encounters an invariant breach: * The facade immediately routes all logging calls to `log_fallback_dispatch`. * Messages format into stack buffers and output directly to `stdout`/`stderr` using `vsnprintf`. * Zero allocations, zero file locks, and zero recursion risk.

Chapter 39. 4. Verification & Testing Matrix

Logging telemetry is continuously verified through 4 test strata: 1. **Functional Tests (`tests/functional/test_log_functional.c`)**: Verifies level formatting, channel bitmask filtering, and prefix formatting. 2. **Unit Tests (`tests/unit/log/test_log_unit.c`, `test_log_engine_unit.c`, `test_log_fallback_unit.c`)**: Tests circular ring buffer overflow, multi-file log rotation, and fallback isolation. 3. **Integrated Tests (`tests/integrated/test_log_integrated.c`)**: Tests concurrent multi-subsystem logging with active simulation events. 4. **System Tests (`tests/system/test_log_system.c`)**: Simulates 10,000 rapid log events, verifying zero memory leaks, file descriptor conservation, and file size bounding.

Chapter 40. Chapter 7: Layer 0 State Debugger & Diagnostic Engine

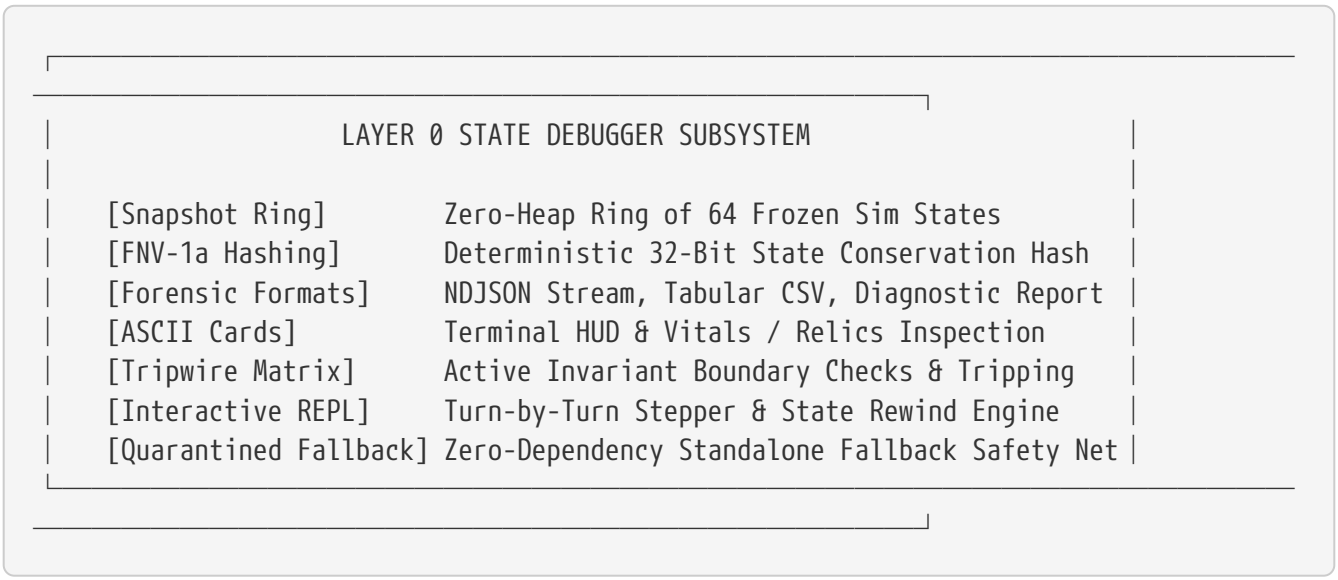
Source File: `/home/professorseanex/Tenebris and Lux/docs/code/debug_system.adoc`

Chapter 41. Overview

This document provides a technical walkthrough and architectural manual for the **Layer 0 State Debugger & Diagnostic Engine Subsystem** (`engine/include/debug/` and `engine/src/debug/`). Serving as the diagnostic instrumentation and state-conservation apparatus of **The Big Machine**, the debugger subsystem provides zero-heap frozen simulation snapshots, deterministic FNV-1a 32-bit state hashing, multi-format forensic exports (Fixed-column, Tabular CSV, Structured NDJSON, Diagnostic Reports), live ASCII inspection cards, an active Invariant Tripwire Matrix, and time-travel state rewind. With strict enforcement of `ALLOC_ZERO_HEAP`, complete reentrancy, and quarantined emergency fallbacks, developers and automated test suites can audit turn-by-turn game state without introducing heisenbugs or simulation drift.

Chapter 42. 1. Architectural Philosophy & Mission

Debugging a tactical RPG requiring absolute simulation determinism cannot rely on intrusive print statements or ad-hoc debugger attachments. The Tenebris and Lux engine implements an **Observable State Machine**:



42.1. Key Architectural Pillars

1. **Decoupled State Representation (`debug_state_fields_t`)**: Layer 0 declares an immutable state representation composed strictly of primitive integers and bounded character buffers. Layer 1 (`sim_state.h`) maps its domain structs into this structure, strictly preserving the downward dependency flow.
2. **Zero-Heap Allocation Guarantee (`ALLOC_ZERO_HEAP`)**: The entire snapshot history (default capacity 64 snapshots) resides in a static ring buffer in the BSS segment. Dynamic allocation is prohibited.
3. **Deterministic FNV-1a State Hashing**: Evaluates a 32-bit Fowler–Noll–Vo prime hash across all state fields at every simulation step, ensuring that identical turns always yield identical checksums.
4. **Multi-Format Forensic Telemetry**: Exports structured state streams in NDJSON (for automated visualizers), CSV (for spreadsheet analysis), and human-readable Diagnostic Reports.
5. **Quarantined Fallback Isolation**: If diagnostics manifests are damaged or file output fails, the quarantined fallback (`debug_fallback.[ch]`) maintains state hashing and console inspection capabilities.

Chapter 43. 2. Subsystem Layout & File Manifest

```
engine/include/debug/
 ├──── debug.h              // Public 5-block interface, types, REPL API & tripwires
 ├──── debug_engine.h       // Internal state snapshot ring, formats & card renderers
 └──── debug_fallback.h     // Quarantined zero-dependency emergency fallback

engine/src/debug/
 ├──── debug.c              // Public facade routing state captures & REPL loop
 ├──── debug_engine.c       // Ring buffer, FNV-1a hasher, multi-format export engines
 └──── debug_fallback.c     // Quarantined emergency fallback implementation

game/data/system/diagnostics/debugger/
 ├──── manifest.json        // Debug catalog and active profile bindings
 ├──── default.json         // Standard developer debugging baseline profile
 ├──── strict.json          // Rigid tripwire policy halting on any invariant breach
 ├──── trace.json           // High-frequency cog-level trace profile
 ├──── snapshot_policy.json // Ring buffer capacity, retention, and rewind limits
 └──── inspection_cards.json// ASCII layout definitions for terminal HUD cards
```

Chapter 44. 3. Core Engine Cogs & API Tour

44.1. 3.1 State Representation & Frozen Snapshot Ring (debug_engine.[ch])

Simulation state is converted into debug_state_fields_t and captured at turn boundaries:

```
typedef struct {
    uint32_t turn_count;
    int32_t  moral_sway;         /* [-100, +100] */
    uint32_t stance;             /* 0: Neutral, 1: Warding, 2: Striking, 3: Eq */
    uint32_t hero_hp;
    uint32_t hero_mp;
    uint32_t hero_tp;
    uint32_t shield_power;       /* [0, 100] */
    uint32_t sword_edge;         /* [0, 100] */
    uint32_t active_enemies;
    uint32_t state_hash_fnv1a;   /* 32-bit checksum */
    uint64_t timestamp_ns;
} debug_state_snapshot_t;

/* Captures a snapshot into the circular ring buffer */
bool debug_engine_record_snapshot(const debug_state_fields_t *fields);

/* Retrieves historical snapshots by relative turn offset (0 = current, 1 = previous)
*/
const debug_state_snapshot_t *debug_engine_get_snapshot(size_t index);
```

44.2. 3.2 Deterministic FNV-1a Hashing

Calculated across the raw byte image of the state fields using the standard FNV-1a offset basis (0x811C9DC5) and prime (0x01000193):

```
uint32_t debug_compute_state_hash(const debug_state_fields_t *fields);
```

Invariants Enforced: * Any state change—whether 1 point of HP or 1 tick of Moral Sway—alters the hash value. * Replays that produce divergent hashes at identical turn indices trigger instant tripwire alarms.

44.3. 3.3 Multi-Format Forensic Exporters

The diagnostic engine provides memory-bounded formatters writing into caller buffers:

```
/* Generates single-line newline-delimited JSON record for stream ingestion */
```

```
size_t debug_format_ndjson(const debug_state_snapshot_t *s, char *out_buf, size_t
max_len);

/* Generates comma-separated tabular record for spreadsheets */
size_t debug_format_csv(const debug_state_snapshot_t *s, char *out_buf, size_t
max_len);

/* Generates human-readable multi-line forensic audit report */
size_t debug_format_report(const debug_state_snapshot_t *s, char *out_buf, size_t
max_len);
```

44.4. 3.4 Live ASCII Inspection Cards

For headless CLI and terminal developers, the engine formats high-density ASCII inspection cards:

```
┌──────────────────────────────────────────────────────────────────────────────
──────────────────────────────────────┐
│ TENEBRIS AND LUX - HERO DOSSIER & COMBAT VITALS                              │
├──────────────────────────────────────────────────────────────────────────────
──────────────────────────────────────┤
│ Turn: 0012 | Stance: EQUILIBRIUM [3] | Sway: +35 (Virtue)    │
│ HP: [▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓] 450/650 | MP: 090/130 | TP: 045/100   │
│ Shield of Lux: 075% [WARD READY] | Sword of Tenebris: 060%   │
│ Active Enemies: 02 | State Hash: 0x8A4F12D9                   │
└──────────────────────────────────────────────────────────────────────────────
──────────────────────────────────────┘
```

44.5. 3.5 Active Invariant Tripwire Matrix

Configured via `game/data/system/diagnostics/taxonomy/tripwire_matrix.json`, tripwires enforce system bounds at runtime:

```
typedef enum {
    TRIPWIRE_HP_BOUNDS,         /* Hero HP within [0, Max HP] */
    TRIPWIRE_SWAY_BOUNDS,       /* Moral Sway within [-100, +100] */
    TRIPWIRE_RELIC_BOUNDS,      /* Shield & Sword power within [0, 100] */
    TRIPWIRE_HASH_INTEGRITY     /* State hash matches expected replayed hash */
} tripwire_id_t;

bool debug_check_tripwire(tripwire_id_t id, const debug_state_fields_t *state);
```

Actions on Breach: * In `strict` profile: immediately aborts simulation tick, emits forensic dump, and halts. * In `default` profile: clamps the out-of-bounds variable, emits `LOG_WARN`, and continues.

44.6. 3.6 Interactive REPL & State Rewind (`--debug`)

When launched with `tenebris_lux --debug`: * The engine pauses between simulation turns, launching a lightweight interactive terminal prompt. * Commands supported: `step`, `inspect`, `history`, `rewind <N>`, `dump <format>`, `quit`. * `rewind <N>` re-hydrates the live Layer 1 simulation directly from historical snapshot `N`.

Chapter 45. 4. Verification & Testing Matrix

The state debugger is continuously validated across 4 test tiers: 1. **Functional Tests (`tests/functional/test_debug_functional.c`)**: Verifies FNV-1a hash distribution, card string formatting, and tripwire evaluations. 2. **Unit Tests (`tests/unit/debug/test_debug_unit.c`, `test_debug_engine_unit.c`, `test_debug_fallback_unit.c`)**: Tests circular ring buffer overflow, state rewind restoration, and fallback isolation. 3. **Integrated Tests (`tests/integrated/test_debug_integrated.c`)**: Integrates debugger captures with live combat actions and stance state transitions. 4. **System Tests (`tests/system/test_debug_system.c`)**: Executes 500 consecutive simulation turns, verifying continuous hash conservation, zero dynamic memory leaks, and snapshot indexing precision.

Chapter 46. Chapter 8: Layer 1 Simulation Engine Code Tour

Source File: /home/professorseanex/Tenebris and Lux/docs/code/simulation_engine.adoc

Chapter 47. Overview

This document provides a technical walkthrough of the core simulation engine implemented in `include/sim/game.h` and `src/sim/game.c`. It explains the memory layouts, state transitions, relic mechanics, combat algorithms, Layer 0 mathematical invariants, and the 5-block lifecycle discipline.

Chapter 48. 1. State Structure & Memory Layout (`player_state_t`)

The heart of the simulation is defined in `include/sim/game.h`:

```
typedef struct {
    int shield_power;           // [0..100] Barrier integrity & Lux charge
    int sword_edge;             // [0..100] Blade sharpness & Tenebris charge
    int moral_sway;             // [-100..+100] Moral alignment axis
    int threat_hp;              // [0..100] Threat vigor of active foe
    combat_stance_t stance;     // Active stance: warding, striking, equilibrium
    location_id_t location;     // Current Kingdom province
    bool is_running;            // Game loop active flag
    int turns_taken;            // Total turns / actions taken
    int foes_vanquished;        // Number of vanguard defeated
    int innocents_warded;       // Number of allies protected
} player_state_t;
```

All state variables are strictly bounded integers with zero dynamic allocations on the heap. This ensures deterministic cache locality, zero memory leaks, and instant serialization.

Chapter 49. 2. Universal Action Dispatcher (`game_process_command`)

Located in `src/sim/game.c`, `game_process_command()` acts as the singular entry point for all player intentions, whether emitted by the terminal CLI or the native GUI:

```
void game_process_command(player_state_t *state, const char *raw_input,
    char *out_msg, size_t msg_len);
```

49.1. Execution Pipeline:

1. **Input Sanitization**: Copies `raw_input` safely via `str_safe_copy()` and executes in-place whitespace trimming (`str_trim_inplace()`).
2. **Verb Tokenization**: Splits input into `command` and trailing `arg`.
3. **Action Dispatch**:
   - `ward`: Calls `handle_ward()`, modifying `shield_power` via `math_saturating_add()`, staggering active threats with `math_saturating_sub()`, and elevating moral virtue with `math_clamp_int()`.
   - `strike`: Calls `handle_strike()`, spending `sword_edge` via `math_saturating_sub()` to inflict heavy severance damage.
   - `balance`: Calls `handle_balance()`, simultaneously recovering both relics, parrying blows, and nudging moral sway toward 0 via `math_nudge_toward()`.
   - `stance`: Calls `handle_stance()`, shifting the tactical state machine.
   - `move`: Calls `handle_move()`, validating directional topology and spawning province encounters.
4. **Invariant Logging**: Writes a human-readable telemetry outcome into `out_msg` for presentation.

Chapter 50. 3. Diagnostic Self-Auditing (`game_audit_state`)

Following Block 4 of the OmniCode protocol, `src/sim/game.c` includes built-in diagnostic audit routines:

```
game_audit_report_t game_audit_state(const player_state_t *state);
int game_run_self_audit(void);
```

Every state boundary (shield power, sword edge, moral sway, stance, location) is checked against compile-time limits. If an invariant is violated, an audit report is generated with exact error messages, enabling automated continuous integration testing without relying on crude process crashes.

Chapter 51. Chapter 9: Layer 2 Presentation Viewport Code Tour

Source File: /home/professorseanex/Tenebris and Lux/docs/code/presentation_gui.adoc

Chapter 52. Overview

This document provides a comprehensive architectural and code walkthrough of the modular Layer 2 Presentation engine implemented in `engine/include/viewport/` and `engine/src/viewport/`. Decomposed into four specialized sub-assemblies (`viewport/raster/`, `viewport/hud/`, `viewport/reel/`, `viewport/styling/`), the system implements the **Movie Reel Architecture**: a continuous cinematic scene film strip traversed via horizontal Push/Pull transitions, with vertical Pop-Out/Sink-In depth kinematics, high fantasy palette blending, and a 60 FPS double-buffered X11 presentation pump.

Chapter 53. 1. The Movie Reel Mental Model & Subfolder Strata

The viewport subsystem is structured into four cohesive sub-assemblies matching its mechanical mental model:

```
engine/
 ├──── include/viewport/
 │     ├──── viewport.h              <-- Master Viewport Umbrella Header
 │     ├──── raster/                 <-- Low-level double-buffered X11 rasterization &
2.5D math
 │     │     ├──── gui.h
 │     │     ├──── gui_types.h
 │     │     └──── gui_view2d.h
 │     ├──── hud/                    <-- Desktop HUD panels, layout bounding boxes, and
action dock
 │     │     ├──── gui_hud.h
 │     │     ├──── gui_layout.h
 │     │     └──── gui_widgets.h
 │     ├──── reel/                   <-- The Movie Reel engine & canonical screen
frames
 │     │     ├──── reel.h
 │     │     └──── gui_screens.h
 │     └──── styling/                <-- Visual palettes, TrueType fonts, and asset
abstraction
 │           ├──── gui_palette.h
 │           ├──── gui_fonts.h
 │           └──── gui_assets.h
 └──── src/viewport/
       ├──── raster/
       ├──── hud/
       ├──── reel/
       └──── styling/
```

Sub-Assembly	Source Stratum	Architectural Mission
`raster/`	`gui.c`, `gui_view2d.c`	Double-buffered X11 context, 2.5D diamond isometric projection, letterbox/pillarbox scalers.
`hud/`	`gui_hud.c`, `gui_layout.c`, `gui_widgets.c`	Desktop HUD, 5 disjoint panels, non-overlapping AABB audits, interactive action dock.
`reel/`	`reel.c`, `gui_screens.c`	The Movie Reel orchestrator: horizontal scene strip (push/pull) and vertical modal depth (pop out/sink in).

Sub-Assembly	Source Stratum	Architectural Mission
`styling/`	`gui_palette.c`, `gui_fonts.c`, `gui_assets.c`	Radiant hope colormaps, TrueType typography (`Cinzel`, `MedievalSharp`), procedural vector art abstraction.

Chapter 54. 2. The Movie Reel: Horizontal Motion & Vertical Depth

The core conceptual paradigm of the Rev 4 presentation engine is the **Movie Reel**:

```
                          POP OUT (Z-Depth / Up)
                          ▲ [Modals, Inspection Cards, Dialogues]
                          |
[Scene 0: TACTICAL] <==== PUSH / PULL ====> [Scene 1: STATUS] <====> [Scene 2:
INVENTORY] ...
(Film Cell Frame)         (Left / Right)     (Film Cell Frame)        (Film Cell
Frame)
                          |
                          ▼ [Terrain Floor, Fog, Sunken Canvas]
                          SINK IN (Z-Depth / Down)
```

54.1. 2.1 Horizontal Axis: Push and Pull (Left and Right)

Scenes and screens are treated as consecutive frames along a physical celluloid reel strip: * **Push Right (`REEL_DIR_PUSH_RIGHT`)**: Advancing forward along the reel to a subsequent screen (e.g., from `GUI_SCREEN_TACTICAL` [F1] to `GUI_SCREEN_STATUS` [F2]). The active scene slides leftward as the new scene arrives from the right. * **Pull Left (`REEL_DIR_PULL_LEFT`)**: Rewinding backward along the reel to a prior screen (e.g., returning from `GUI_SCREEN_STATUS` [F2] back to `GUI_SCREEN_TACTICAL` [F1]). The active scene slides rightward as the prior scene is pulled back in. * **Smooth-Step Kinematics**: Horizontal transition progress is evaluated using normalized Hermite interpolation ($S(t) = 3t^2 - 2t^3$) scaled to the viewport width, guaranteeing fluid motion without visual tearing.

54.2. 2.2 Vertical Axis: Pop Out and Sink In (Up and Down)

Within any active scene on the reel, visual elements possess continuous Z-elevation depth: * **Pop Out (`REEL_DEPTH_POPPED = +1`)**: Elements elevate upward toward the viewer ($\Delta Y = -16\text{ px}$, scale $1.05\times$, full 1.0 opacity, drop shadow). Used for dialogue boxes, relic inspection modals, victory fanfares, and action tooltips. * **Sink In (`REEL_DEPTH_SUNKEN = -1`)**: Elements submerge downward into the background canvas ($\Delta Y = +12\text{ px}$, scale $0.95\times$, attenuated 0.60 opacity, ambient recess). Used for background terrain tiles, ambient fog layers, and sunken meters when a modal pops out. * **Surface (`REEL_DEPTH_SURFACE = 0`)**: The neutral baseline scene plane ($\Delta Y = 0$, scale 1.0, alpha 1.0).

Chapter 55. 3. The 2.5D Tactical Isometric Engine

The tactical battlefield features dual visual projection modes toggleable via `[M]`: * `GUI_VIEW_2D_FLAT`: Classic orthographic top-down grid (38×38 px cells) for overworld travel. * `GUI_VIEW_25D_ISOMETRIC`: Diamond dimetric projection (48×24 px tiles) with vertical Z-axis elevation prisms (8 px per elevation level) for tactical skirmishes.

55.1. Isometric Projection & Unprojection Reciprocity:

$$\text{screen_x} = X_0 + (gx - gy) \cdot \frac{W}{2}$$

$$\text{screen_y} = Y_0 + (gx + gy) \cdot \frac{H}{2} - gz \cdot E$$

Unprojecting mouse clicks (sx, sy) on ground plane gz back to discrete grid (gx, gy):

$$gx = \left\lfloor \frac{sx - X_0}{W} + \frac{sy + gz \cdot E - Y_0}{H} + 0.5 \right\rfloor$$

$$gy = \left\lfloor \frac{sy + gz \cdot E - Y_0}{H} - \frac{sx - X_0}{W} + 0.5 \right\rfloor$$

Chapter 56. 4. Non-Anonymous Type Invariant Covenant

In strict compliance with engine architectural invariants, **all structs and enums** throughout the viewport and reel strata are explicitly tagged: * `typedef enum gui_screen_mode { … } gui_screen_mode_t;` * `typedef enum reel_direction { … } reel_direction_t;` * `typedef enum reel_state { … } reel_state_t;` * `typedef enum reel_depth { … } reel_depth_t;` * `typedef struct reel_depth_element { … } reel_depth_element_t;` * `typedef struct reel { … } reel_t;` * `typedef struct gui_palette { … } gui_palette_t;` * `typedef struct gui_layout { … } gui_layout_t;` * `typedef struct gui_button { … } gui_button_t;`

Chapter 57. 5. Verification Telemetry

The presentation stratum is verified by `tests/unit/test_gui_unit.c` and `tests/unit/viewport/test_reel_unit.c` across the four ontological testing tiers: 1. **Section 1 (Always Pass)**: Baseline layout bounds, reel default state, and 9-button command dock. 2. **Section 2 (Never Fail)**: Out-of-bounds click safety, negative duration clamping, and element capacity limits. 3. **Section 3 (Axioms & Tripwires)**: AABB panel disjointness, projection reciprocity, push/pull kinematics, and pop-out/sink-in elevation vectors. 4. **Section 4 (Emergence)**: Full `gui_audit_layout()` and `reel_audit()` preflight execution.

Chapter 58. Chapter 10: Layer 0/2 Audio Event Bus & Mixer Subsystem

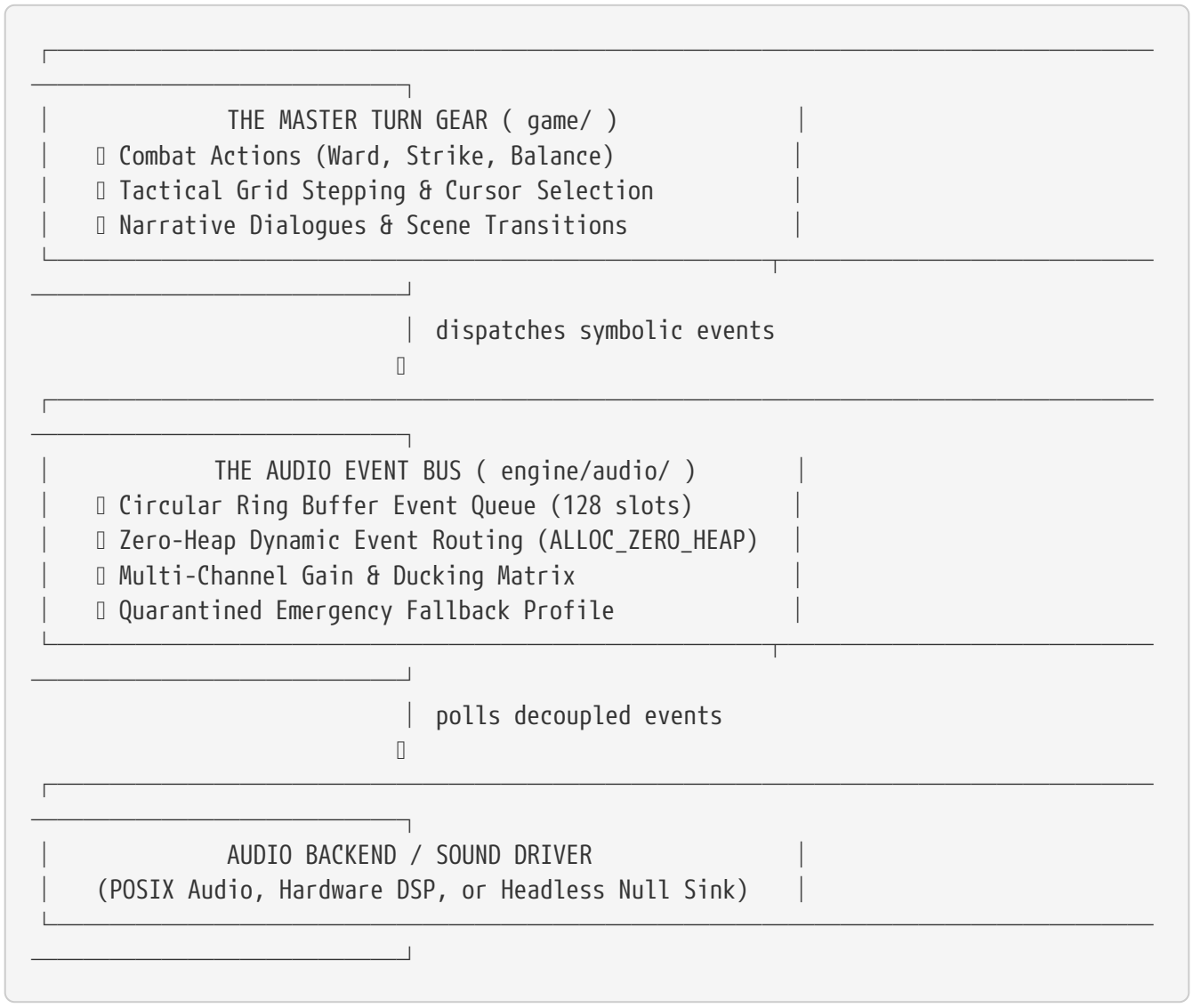
Source File: /home/professorseanex/Tenebris and Lux/docs/code/audio_subsystem.adoc

Chapter 59. Overview

This document provides a comprehensive architectural and code walkthrough of the dedicated Audio subsystem implemented in `engine/include/audio/` and `engine/src/audio/`. Operating as an essential stratum of **The Big Machine**, the audio subsystem provides a multi-channel mixing bus (Master, BGM, Ambience, Combat SFX, Tactical SFX, UI, Stingers), a zero-heap circular ring buffer event queue, bounded volume attenuation, high-priority ducking matrices, and a quarantined monolithic fallback profile.

Chapter 60. 1. Architectural Philosophy: The Decoupled Audio Machine

In traditional game engines, audio drivers are often tightly coupled to simulation loops or low-level OS sound APIs (ALSA, PulseAudio, WASAPI). In **Tenebris and Lux**, the audio architecture strictly obeys the **Machine vs. Master Turn Gear Separation**:



60.1. Axiomatic Invariants:

1. **Zero Dynamic Heap Allocations (`ALLOC_ZERO_HEAP`)**: The event bus allocates zero bytes on the heap; all queues, channels, and cue manifests reside in static, reentrant memory.
2. **Read-Only Simulation Influence**: Audio strictly follows simulation events. Audio playback never gates, blocks, or alters simulation state transitions or turn resolution.
3. **Hardware Limiter & Bounded Gain**: All channel volume levels are strictly clamped to $[0.0, 1.0]$.
4. **All Structs and Enums Explicitly Named**: Zero anonymous structs or enums.

Chapter 61. 2. Subsystem File Layout & Header Structure

The audio subsystem is cleanly partitioned into public interfaces, implementations, and fallback:

```
engine/
├──── include/audio/
│    ├──── audio.h              <-- Master Audio Umbrella Header
│    ├──── audio_types.h        <-- Channel enums, event packets, cue manifests,
stats
│    ├──── audio_bus.h          <-- Public event posting, polling, channel volume API
│    └──── audio_fallback.h     <-- Quarantined emergency fallback profile header
└──── src/audio/
     ├──── audio_bus.c          <-- Circular ring buffer, channel gain, event routing
     └──── audio_fallback.c     <-- Monolithic emergency fallback profile
implementation
```

Chapter 62. 3. Multi-Channel Mixer Architecture

The audio bus implements seven canonical mixing channels matching `game/data/presentation/audio/channels.json`:

Channel Enum	Canonical Channel Name	Default Gain	Role & Attenuation Behavior
`AUDIO_CHANNEL_MASTER`	Master Output Bus	100%	Controls overall presentation master gain.
`AUDIO_CHANNEL_BGM`	Background Music	75%	Loopable orchestral streams; ducked under stingers.
`AUDIO_CHANNEL_AMBIENCE`	Soundscapes & Ambience	60%	Environmental biome loops (wind, meadow hum, dunes).
`AUDIO_CHANNEL_SFX_COMBAT`	Combat Sound Effects	90%	Strikes, spell chants, Lux Barrio barrier wards.
`AUDIO_CHANNEL_SFX_TACTICAL`	Tactical Grid Audio	80%	Footsteps, tile selection, zone-of-control pings.
`AUDIO_CHANNEL_UI`	User Interface	85%	Button clicks, tab switches, stance toggle cues.
`AUDIO_CHANNEL_STINGER`	Fanfares & Stingers	95%	Victory stingers, battle intro fanfares (ducking source).

Chapter 63. 4. Non-Anonymous Ontological Types

In strict conformance with project governance standards, all data structures and enumerations are explicitly named:

```
typedef enum audio_channel {
    AUDIO_CHANNEL_MASTER = 0,
    AUDIO_CHANNEL_BGM,
    AUDIO_CHANNEL_AMBIENCE,
    AUDIO_CHANNEL_SFX_COMBAT,
    AUDIO_CHANNEL_SFX_TACTICAL,
    AUDIO_CHANNEL_UI,
    AUDIO_CHANNEL_STINGER,
    AUDIO_CHANNEL_COUNT
} audio_channel_t;

typedef enum audio_event_type {
    AUDIO_EVENT_NONE = 0,
    AUDIO_EVENT_PLAY_SFX,
    AUDIO_EVENT_STOP_SFX,
    AUDIO_EVENT_PLAY_BGM,
    AUDIO_EVENT_STOP_BGM,
    AUDIO_EVENT_CROSSFADE_BGM,
    AUDIO_EVENT_PLAY_AMBIENCE,
    AUDIO_EVENT_STOP_AMBIENCE,
    AUDIO_EVENT_CROSSFADE_AMBIENCE,
    AUDIO_EVENT_PLAY_STINGER,
    AUDIO_EVENT_SET_VOLUME,
    AUDIO_EVENT_MUTE_CHANNEL,
    AUDIO_EVENT_STOP_ALL
} audio_event_type_t;

typedef struct audio_event {
    audio_event_type_t type;
    audio_channel_t channel;
    char cue_id[AUDIO_MAX_CUE_ID_LEN];
    float volume;
    float pan;
    float pitch;
    float fade_duration_sec;
    bool loop;
    uint64_t timestamp_us;
} audio_event_t;
```

Chapter 64. 5. Verification Telemetry

The audio subsystem is validated by two dedicated unit test harnesses: * `tests/unit/audio/test_audio_bus_unit.c`: Validates channel default gains, null guards, volume clamping, 128-event ring buffer capacity, FIFO queue order preservation, and high-watermark telemetry. * `tests/unit/audio/test_audio_fallback_unit.c`: Validates the quarantined emergency fallback profile, provenance tagging (`loaded_from_data == false`), and fallback profile application to the live bus.

Chapter 65. Chapter 11: Layer 4 Verification & Test Suite Code Tour

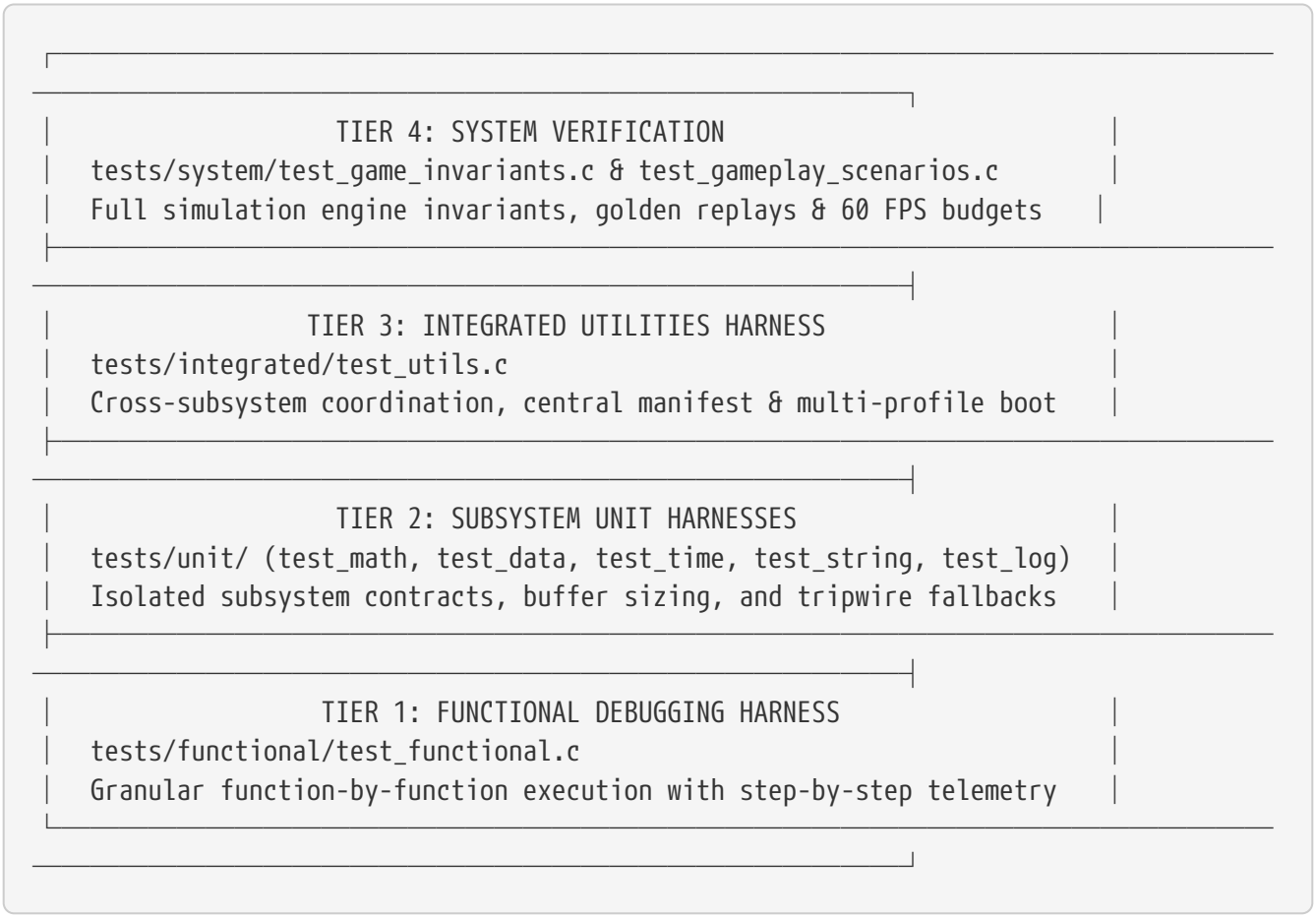
Source File: /home/professorseanex/Tenebris and Lux/docs/code/test_suite.adoc

Chapter 66. Overview

This document provides a comprehensive technical walkthrough of the automated game development verification architecture in **Tenebris and Lux**. Organized into strict hierarchical strata across `tests/`, the test suite mirrors the Layer 0-4 engine ontology and enforces four distinct ontological verification sections within every test harness:

1. **Always Pass (Nominal Golden Baseline)**: Verifies nominal parameters, default asset loading, and canonical execution paths.
2. **Never Fail (Fault-Tolerant Tripwires)**: Injects boundary breaches, NULL pointers, missing files, and corrupt states to prove that the engine never crashes or invokes undefined behavior.
3. **Assertion & Guarantee (Axiomatic Invariants)**: Mathematical guardrails, energy conservation, monotonic clocks, and DAG dependencies guaranteed by engine contracts.
4. **Emergence (Stochastic Stress & Chained Pipelines)**: Emergent combinatorial state spaces, 1,000 chaotic command fuzzing, bitwise golden replay determinism, and sub-50-microsecond simulation latency budgets.

Chapter 67. 1. The Four-Tier Testing Hierarchy



67.1. Tier 1: Functional Debugging (`tests/functional/`)

Designed specifically for interactive debugging and pinpointing anomalies in isolated functions. Emits explicit `[DEBUG-TRACE]` entries showing arguments before and after execution for each individual cog (e.g. `math_clamp_int`, `math_saturating_add`, `str_safe_copy`, `str_trim_inplace`, `json_get_string`, `combat_get_stance_name`).

67.2. Tier 2: Subsystem Unit Harnesses (`tests/unit/`)

Provides 1:1 test coverage for each dedicated Layer 0 utility subsystem folder: * **`test_math.c`**: Bounded arithmetic, saturating additions/subtractions, step-bounded equilibrium nudging, proportional ratio scaling, 2D discrete AABB hit-testing, and `data/math/default.json` profile loading. * **`test_data.c`**: Zero-heap filesystem I/O (`fs_utils.c`), canonical buffer sizing (`FS_BUFFER_SMALL_BYTES` = 1024 to `FS_BUFFER_MAX_CONFIG` = 65536), bounded file reading/writing, scoped JSON object querying, and array iteration. * **`test_time.c`**: Monotonic millisecond/microsecond clocks, POSIX nanosleep frame pacing, ISO 8601 timestamp generation ("YYYY-MM-DDTHH:MM:SSZ"), and 60 FPS / 120 FPS profile loading. * **`test_string.c`**: Buffer-safe bounded copying, in-place whitespace trimming, case-insensitive equality/substring matching, parent directory extraction, and duplicate-slash immune path joining. * **`test_log.c`**: Forensic telemetry, graded severity filtering, categorized logging channels, in-memory static ring buffer, and emergency monolithic code page fallbacks. *

`test_debug.c`: Interactive state debugger, frozen snapshot ring buffer, FNV-1a checksum determinism, time-travel state rewind, state diffing, dynamic breakpoints, and emergent chaotic stress.

67.3. Tier 3: Integrated Utilities Harness (`tests/integrated/`)

- `test_utils.c`: Validates cross-subsystem coordination of all Layer 0 utilities. Exercises the central manifest registry (`manifest.c`), validates upstream DAG dependency resolution, enforces diagnostic escalation policies (WARNING → `PREFLIGHT_STATUS_WARNINGS`, CRITICAL → `PREFLIGHT_STATUS_DOWNGRADED`, FATAL → `PREFLIGHT_STATUS_HALT_FATAL`), and benchmarks multi-profile synchronous boot latency (< 5 milliseconds).

67.4. Tier 4: System Verification (`tests/system/`)

- `test_game_invariants.c`: Validates Layer 1 simulation engine state invariants: twin relic power bounds ($[0, 100]$), moral sway continuum boundaries ($[-100, +100]$), protagonist name customization, and bidirectional provincial map topology.
- `test_gameplay_scenarios.c`: Simulates end-to-end tactical combat routes (Righteous Warden and Umbral Striker paths), 15-step golden deterministic replay parity, 1,000 chaotic command fuzzing for softlock immunity, and 10,000-turn simulation latency benchmarking (< 50 microseconds per turn).

Chapter 68. 2. The Four Ontological Testing Sections

Every test harness in the repository implements the four-section ontological model:

Section	Category	Mission & Verification Scope
Section 1	**Always Pass**	Valid inputs, nominal parameters, default configuration loading, standard state initialization, well-formed JSON, normal gameplay actions.
Section 2	**Never Fail**	Inviolability under error. Injects NULL pointers, zero capacities, missing files, corrupted JSON, and unknown stances. Proves tripwire activation and safe fallback without OS traps.
Section 3	**Assertion & Guarantee**	Axiomatic mathematical laws and structural contracts: bounded clamping, non-decreasing monotonic clocks, strict null-termination under truncation, and deterministic stance completeness.
Section 4	**Emergence**	Higher-order systemic behaviors: 1,000 pseudo-random fuzzing iterations, spatial grid overlap emergence, golden replay bitwise equality across runs, and < 50 us latency budgets.

Chapter 69. 3. CMake & CTest Test Targets

The build configuration registers twelve automated test targets within `CMakeLists.txt`:

```
# CLI Interface Contract Verification Tests
add_test(NAME cli_help_test COMMAND tenebris-lux --help)
add_test(NAME cli_version_test COMMAND tenebris-lux --version)
add_test(NAME subsystem_audit_test COMMAND tenebris-lux --audit)

# Tier 1: Functional Debugging Harness
add_test(NAME functional_cogs_test COMMAND test_functional)

# Tier 2: Subsystem Unit Harnesses (Isolated Subsystems)
add_test(NAME unit_math_test COMMAND test_math)
add_test(NAME unit_data_test COMMAND test_data)
add_test(NAME unit_time_test COMMAND test_time)
add_test(NAME unit_string_test COMMAND test_string)
add_test(NAME unit_log_test COMMAND test_log)
add_test(NAME unit_debug_test COMMAND test_debug)

# Tier 3: Integrated Utilities Harness (Multi-Subsystem Coordination)
add_test(NAME integrated_utils_test COMMAND test_utils)

# Tier 4: System Verification Harnesses (Full Game Simulation)
add_test(NAME system_game_invariants_test COMMAND test_game_invariants)
add_test(NAME system_gameplay_scenarios_test COMMAND test_gameplay_scenarios)
```

Chapter 70. 4. Execution & Verification Commands

Developers and CI/CD pipelines can invoke the complete suite or isolate specific tiers:

```
# Compile all binaries and test targets under strict zero-warning policy
cmake --build build --config Debug

# Execute all 13 automated verification suites
ctest --test-dir build -C Debug --output-on-failure

# Run individual tier harnesses directly for step-by-step debugging
./build/test_functional         # Granular cog trace output
./build/test_math               # Math & geometry unit verification
./build/test_data               # Filesystem & JSON unit verification
./build/test_time               # Timing & clock unit verification
./build/test_string             # Safe string unit verification
./build/test_log                # Forensic telemetry unit verification
./build/test_debug              # State debugger & rewind unit verification
./build/test_utils              # Integrated utilities verification
./build/test_game_invariants    # Simulation state invariants
./build/test_gameplay_scenarios # Gameplay scenarios & latency benchmark
```

Chapter 71. Chapter 12: Technical Specification: Data-Driven Architecture & Manifest System

Source File: /home/professorseanex/Tenebris and Lux/docs/architecture/data_driven_architecture.adoc

Chapter 72. Abstract & Architectural Mandate

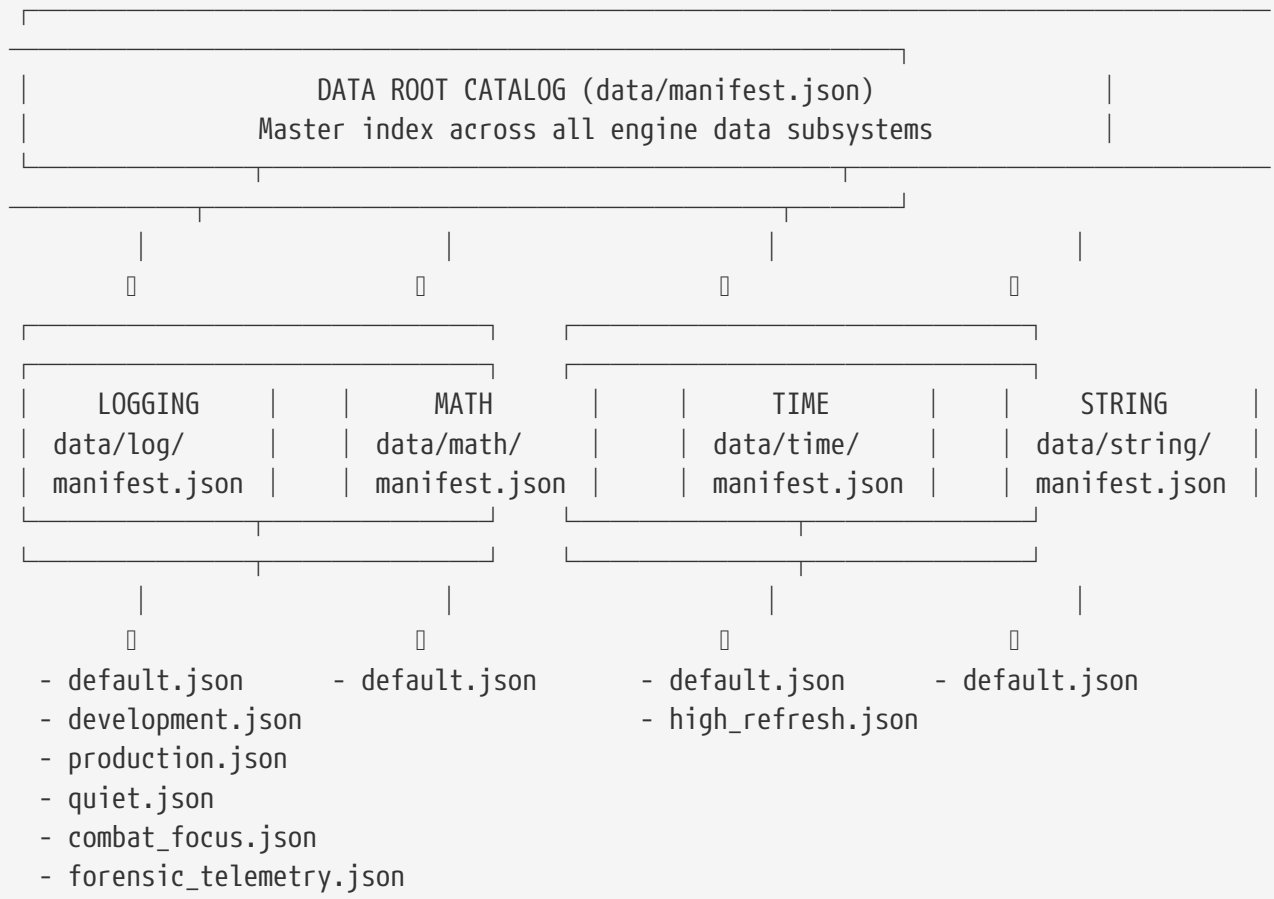
This specification establishes the **Data-Driven Subsystem** for **Tenebris and Lux**. Transitioning from hardcoded compile-time values to externalized, validated data files, this architecture enforces the **"Manifest as Main Menu" and "Files as Settings"** ontological pattern. This pattern enables live configuration hot-swapping, deterministic testing, and seamless integration between C11 Layer 0 engine utilities and high-level tools. It encompasses logging profiles (`data/log/`), mathematical continuum boundaries (`data/math/`), frame pacing budgets (`data/time/`), and lexical token disciplines (`data/string/`).

Chapter 73. 1. System Philosophy: Manifest as Main Menu

73.1. 1.1 The Menu-to-Settings Ontological Pattern

Traditional game configuration often conflates cataloguing with concrete configuration. Under the **Tenebris and Lux** architecture, a clean structural boundary is maintained:

- **The Manifest (`manifest.json`)**: Operates as the **"Main Menu"**. It catalogs, describes, indexes, and exposes selectable configuration entries (`menu_items`). It enforces invariants governing which settings are available, provides default selections, and maps hotkey shortcuts.
- **The Data Files (`*.json`)**: Operate as the concrete **"Settings"**. Each file represents a fully-formed, isolated configuration state conforming directly to the underlying engine structures (e.g., `log_config_t`, `math_config_t`, `time_config_t`, `string_config_t`).



73.2. 1.2 OmniCode 5-Block Conformance in JSON Serialization

In accordance with the repository standard, every JSON data asset strictly adheres to the **Universal 5-Block Lifecycle Protocol**:

1. **Block 1:** `_metadata`: Identity tags (`@project`, `@file`, `@id`, `@version`, `@author`, `@invariants`).
2. **Block 2:** `_types`: Schema specification, target C struct bindings, enum sets.
3. **Block 3:** `menu` / `profile` / `setting`: Operational payload containing menu entries or profile parameters.
4. **Block 4:** `_audit`: Diagnostic limits, rotation caps, and memory footprint contracts (`ALLOC_ZERO_HEAP`).
5. **Block 5:** `_navigation`: Upstream parent manifest links and fallback targets.

Chapter 74. 2. Active Data-Driven Subsystems

74.1. 2.1 Logging & Telemetry (`data/log/`)

Defines six distinct configuration profiles (`default`, `development`, `production`, `forensic_telemetry`, `quiet`, `combat_focus`) controlling min severity, console ANSI echoing, file rotation limits, and telemetry tracking categories.

74.2. 2.2 Mathematical Continuum (`data/math/`)

Defines canonical bounds and continuum limits governing game metaphysics:

- **Resonance Limits**: `[0, 100]` strictly preventing relic over-saturation or underflow.
- **Moral Sway Continuum**: Symmetric `[-100, +100]` with equilibrium anchor at `0`.
- **Equilibrium Stance Band**: `[-25, +25]` threshold for dual relic synergy.
- **Overload / Eclipse Thresholds**: `+75` (Radiance) and `-75` (Abyssal Eclipse).
- **Viewport Dimensions**: Default desktop presentation bounds (`800 x 600`).

74.3. 2.3 Monotonic Timing & Frame Pacing (`data/time/`)

Defines latency budgets and frame sleep targets:

- `default.json`: 60 FPS profile (`16.666667 ms` frame budget, `50.0 us` simulation turn budget).
- `high_refresh.json`: 120 FPS profile (`8.333333 ms` frame budget, `25.0 us` simulation turn budget).

74.4. 2.4 Lexical Buffers & Token Dictionary (`data/string/`)

Defines textual safety limits and canonical identifiers:

- **Buffer Capacities**: Max path (512 B), max name (64 B), max command (128 B), line discipline (80 chars).
- **Stance Tokens**: `NEUTRAL`, `WARDING`, `STRIKING`, `EQUILIBRIUM`.
- **Action Verbs**: `help`, `version`, `audit`, `cli`, `gui`, `barrio`, `cuttatio`, `duo`, `stance`, `journey`, `inspect`, `name`, `quit`.

Chapter 75. 3. The Fallback Tripwire Protocol

A central invariant of the data-driven architecture is **Fault-Tolerant Determinism**:

1. **Attempt Data Load**: Subsystems first attempt to deserialize settings from their respective `data/` JSON file.
2. **Tripwire Detection**: If the path does not exist, file permissions fail, or JSON formatting is invalid, the engine detects this condition immediately without crashing or memory corruption.
3. **Hardcoded Fallback Activation**: The subsystem loads its compile-time fallback baseline (e.g. `math_get_default_config()`, `time_get_default_config()`, `string_get_default_config()`, `log_get_default_config()`).
4. **Diagnostic Telemetry**: A warning trace is emitted to the diagnostic log detailing the fallback event and preserving system stability.

Chapter 76. 4. Verification & Diagnostic Testing

All data files in `data/` and its subsystem folders are validated for syntactic correctness, schema compliance, and referential integrity by CTest suites (`test_utils`, `test_log`, `test_game_invariants`) and developer tooling (`tl_tools`).